∞ Causal Generation, Retrospective Consolidation: Completed-Segment Bidirectional Memory Without Temporal Leakage

MMLA Focus Paper

Junyi Zou[1,*] • **Avrova Donz**[1,2,*]

[1]MMLA-org [2]Communication University of China (CUC), No. 1 Dingfuzhuang East Street, Chaoyang District, Beijing 100024, China

[*]Equal contribution

Email: zoujunyi@zjydiary.cn · avrovadonz@icloud.com

Project: github.com/MMLA-org/mmla-memory

ABSTRACT

Autoregressive generation and bidirectional inspection need not conflict when they occur on opposite sides of an explicit boundary. We define *completed-segment bidirectional consolidation* (CSBC): tokens are generated causally and receipted immutably; only after a bounded segment closes may a retrospective encoder inspect that already observed view in both directions, eventize it, and propose later authoritative-memory updates. Every accepted event receives a logical order and an exposure point strictly after closure. It may influence only later reads. It cannot rewrite a historical logit, random draw, emitted token, memory-read receipt, or audit record.

The contract separates five clocks—token time, boundary index, ordered event time, training-supervision time, and wall-clock service time—so worker completion order never becomes semantic order. We type fixed and adaptive cores, overlap, bounded carry, anchors, idempotence keys, candidate rows, exact NULL, pinned versions, queues, ledgers, and resource vectors. Under explicit immutable-history, stopping-time, unique-ownership, deterministic-eventizer, idempotence, atomic-publication, and bounded-service assumptions, we prove no retroactive effect, a prefix-twin noninterference hyperproperty, teacher-branch amputation, exactly-once emission, event-level locality, serializability, conditional commutation, bounded overlap/compute/memory, and deterministic queue stability. Counterexamples show how suffix-dependent boundaries, missing anchors, unstable identifiers, unbounded carry, shared-capacity reads, and speculative state reuse invalidate those results.

The manuscript gives asynchronous exposure and recovery rules, full compute/memory/traffic/archive/retry/latency/energy accounting, security invariants, a worked correction timeline, and staged falsification protocols. It reports no experiment. It does not claim that a same-checkpoint semantic interface is qualified, that event extraction is correct, that predictive overwrite works, that CSBC is strict RTT, or that any quality–cost crossover has been measured. CSBC alone is reasoning-time memory adaptation.

1 Introduction

1.1 Bidirectionality belongs after closure

An autoregressive generator must choose token X_t without reading tokens that do not yet exist. A retrospective encoder can nevertheless inspect a finite block in both directions once every token in that block is already emitted. The apparent contradiction disappears only when the segment boundary, encoding start, commit order, and later exposure are explicit.

DEFINITION

Definition 1.1 (Completed-segment bidirectional consolidation)**.** Completed-segment bidirectional consolidation (CSBC) is a protocol in which (i) token generation through a declared boundary is causal; (ii) the completed bounded view is constructed only after that boundary; (iii) a retrospective encoder may inspect only that completed view bidirectionally; (iv) each derived event produces at most one complete authoritative-row commit or exact NULL; and (v) every resulting version is first readable at an exposure point strictly after the boundary. Historical emissions and receipts are immutable.

The phrase *retrospective encoding* names step (iii). It does not imply reverse-time generation, future-token access, retrospective gradient updates during deployment, or correction of already emitted text. A later answer can use a committed correction; the earlier mistaken sentence remains part of history.

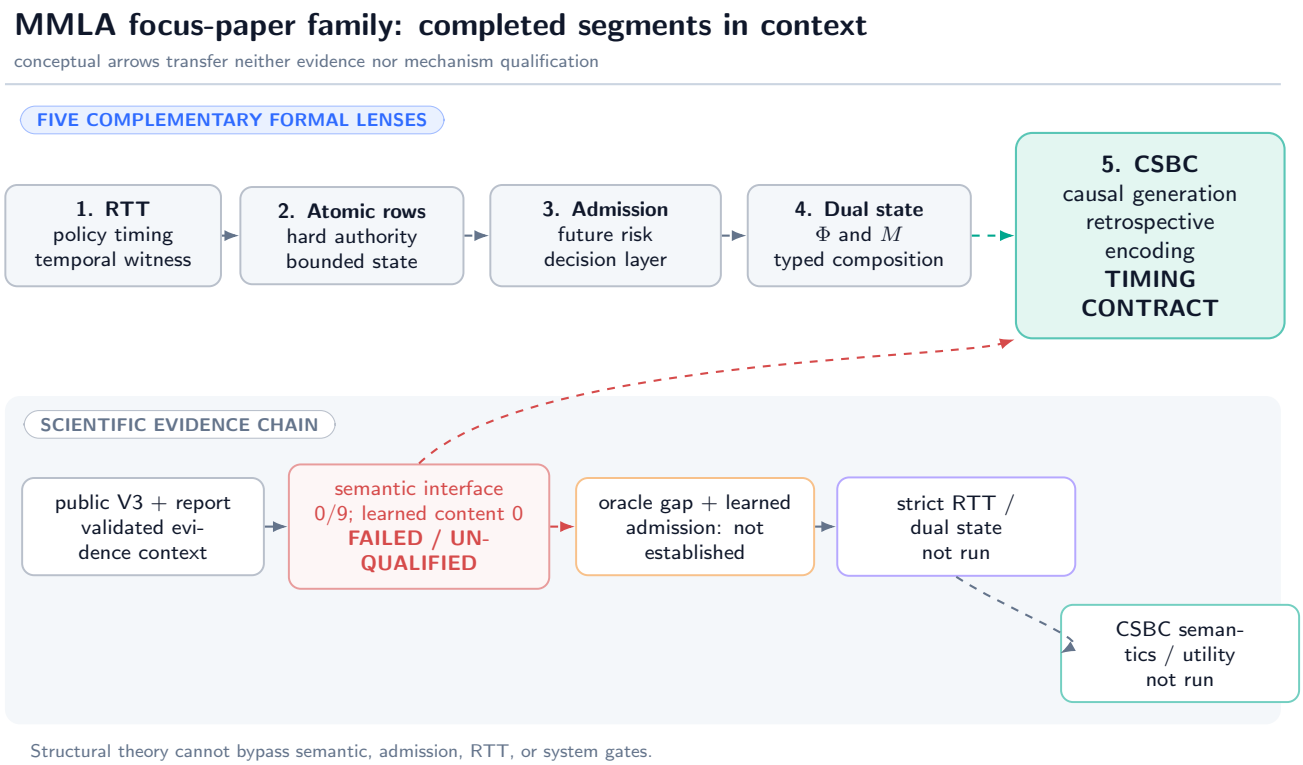

Figure 1: **Completed-segment consolidation in the five-paper conceptual family.** Dashed arrows carry concepts only. The same-checkpoint semantic interface remains failed/unqualified, and no oracle gap, learned admission policy, strict-RTT result, or CSBC utility result is inherited.

1.2 Evidence boundary

Public MMLA V3 and its complete technical-report evidence record provide context [12, 13]. Their admitted boundary remains negative and diagnostic: PM-I2 qualified 0/9 same-checkpoint interfaces; in the fitted-population diagnostic, learned content was exact on 0/73,728 probes both with learned and mechanical target routing, while only the complete mechanical oracle reached 73,728/73,728. Those counts do not evaluate CSBC, but they forbid treating a mechanically well-timed row as a semantically useful one.

The four companion focus papers provide terminology for strict RTT, complete rows, predictive admission, and typed dual state [15, 10, 11, 14]. None transfers empirical validation. In particular:

- CSBC changes authoritative memory, so its later-read path is RTMA;
- it is strict RTT only if a separately evidenced feedback-driven policy update is later read on the same unresolved problem;
- a valid event candidate is not a qualified semantic interface;
- an expected-risk admission rule is neither run nor assumed here; and
- no clean build or conditional theorem reopens a failed empirical gate.

1.3 Contribution and nonclaims

This manuscript owns a timing and systems contract: typed boundaries, five clocks, filtration, exposure, teacher amputation, overlap/carry, event ownership, idempotence, event recursion, asynchronous consistency, resource bounds, information-flow restrictions, failure recovery, and falsification obligations. It does not own a successful event detector, semantic encoder, factual verifier, admission model, storage engine, policy updater, or deployed system.

Formal statements carry their assumptions locally. Definitions, proofs, design prescriptions, planned tests, and unvalidated mechanisms are marked as such; no label promotes an implementation or empirical claim.

CONJECTURED A post-closure bidirectional view may help some eventizers represent local corrections or relations. The conjecture remains open. This paper establishes only what would have to be true for such a mechanism to be causal, bounded, auditable, and falsifiable.

2 Typed Process and Carrier Inventory

2.1 Probability space and causal tokens

Fix a probability space $(\Omega, \mathcal{F}, \mathbb{P})$, token alphabet $\mathcal{X}$, and token filtration $(\mathcal{F}_t^X)_{t\geq 0}$. At token time t, a frozen deployment generator with parameters $\theta \in \mathcal{P}_{\text{dep}}$ reads a pinned authoritative-memory version ν_t and draws

$$L_t = G_\theta(X_{<t}, \text{Read}(M^{\nu_t}), \gamma_t), \tag{1}$$
$$X_t = S(L_t, \xi_t), \tag{2}$$

where γ_t is declared causal workspace and $\xi_t \in \mathcal{U}$ is a recorded random coin. The immutable emission receipt is

$$\rho_t^X = \text{Hash}(\textsf{episode}, t, \text{Hash}(X_{<t}), \text{Hash}(L_t), \xi_t, X_t, \nu_t, \text{Hash}(\gamma_t)). \tag{3}$$

The receipt may store digests rather than unrestricted private reasoning. It must nevertheless bind the causal inputs needed by the claimed reproducibility boundary.

2.2 Segment and event types

Let boundaries be $0 = b_0 < b_1 < \cdots$. Core $I_n = (b_{n-1}, b_n] \cap \mathbb{N}$ has length $C_n = |I_n|$. With overlap $O_n \subseteq \{1, \ldots, b_{n-1}\}$ and carry $q_{n-1} \in \mathcal{Q}$, the completed view is

$$V_n = \text{View}(X_{O_n}, X_{I_n}, q_{n-1}; \beta_n) \in \mathcal{V}, \tag{4}$$

constructed only after X_{b_n} and $\rho_{b_n}^X$ exist. Boundary rule $\beta_n \in \mathcal{B}$ is either fixed or an adapted stopping rule specified in Section 6.

A retrospective encoder $B_\psi : \mathcal{V} \to \mathcal{H}$ may attend bidirectionally within V_n. The eventizer returns

$$(E_n, q_n) = \text{Eventize}(B_\psi(V_n), q_{n-1}; \zeta_n), \quad E_n = (e_{n,1}, \ldots, e_{n,K_n}), \quad K_n \leq K_{\max}, \tag{5}$$

where ζ_n is recorded randomness and q_n has a fixed maximum serialization size B_Q. Each event has typed span, content proposal, anchor $\alpha(e) \in \mathbb{N}$, ownership core, source receipts, and idempotence key

$$\iota(e) = \text{Hash}(\textsf{episode}, \textsf{eventizer-build}, \textsf{normalized-span}, \alpha(e), \textsf{event-type}). \tag{6}$$

2.3 Authority, service, and training objects

Authoritative memory is a fixed table $M \in \mathcal{M} = \mathcal{R}^S$. For event (n, j) and current version $M_{n,j-1}$, a proposer returns a complete candidate or failure,

$$\widehat{r}_{n,j} = \text{Construct}(e_{n,j}, M_{n,j-1}; \omega_{n,j}) \in \mathcal{R} \cup \{\perp\}. \tag{7}$$

Trusted validation and publication choose one complete-row replacement or exact NULL. Tenant, ACL, versions, protection, provenance, tombstone validity, rollback lineage, and final receipt are system-owned fields, never minted by completed text.

Wall-clock service uses a bounded queue $Q_\kappa^{\text{svc}} \in \mathcal{K}$, worker records W_κ^{svc}, pinned predecessor versions, retry records, and a commit ledger $\mathcal{L}^M$. Training-only objects live in a disjoint type $\mathcal{T}$: future branches $F_{g,k}$, labels Y^{T}, teacher risks, gradients, split identifiers, and supervision time s. No value in $\mathcal{T}$ is a deployment input.

Object	Type	Role	Mandatory boundary
$X_t, L_t, \xi_t, \rho_t^X$	$\mathcal{X}, \mathbb{R}^{\lvert\mathcal{X}\rvert}, \mathcal{U}, \mathcal{R}_{\text{emit}}$	causal emission and immutable receipt	fixed before future token exists
I_n, O_n, V_n	finite spans / $\mathcal{V}$	core, overlap, completed view	bounded; V_n only after b_n
β_n, q_n	$\mathcal{B}, \mathcal{Q}$	boundary rule and carry	stopping-measurable; $\lvert q_n\rvert \leq B_Q$
$E_n, e_{n,j}$	$\mathcal{E}^{\leq K_{\max}}, \mathcal{E}$	ordered events	deterministic order for fixed inputs/coins
$\alpha(e), \iota(e)$	token index, digest	ownership and retry identity	stable across overlap and retry
$\widehat{r}$, NULL, M^ν	row/action/memory	candidate, identity action, authoritative state	trusted assembly; atomic publication
θ, ψ	deployment parameters	generator and retrospective encoder	frozen before evaluated episode
F, Y^{T}, s	$\mathcal{T}$	training-only teacher/supervision	amputated from deployment graph
$Q^{\text{svc}}, \mathcal{L}$	bounded queue / ledger	async work and receipts	logical order independent of finish order
$\mathbf{C}$	$\mathbb{R}_+^d$	resource vector	no hidden scalarization

Table 1: Typed carrier inventory. Co-location in one process does not erase causal, authority, lifetime, or accounting distinctions.

2.4 Core assumption ledger

Assumption 2.1 (Causal generation and immutable history)**.** Equations (1)–(2) are $\mathcal{F}_t^X$-measurable. Once ρ_t^X is sealed, L_t, X_t, ξ_t, ν_t, and every earlier receipt are append-only and cannot be modified by the consolidator, queue, committer, or recovery service.

Assumption 2.2 (Post-closure construction and exposure)**.** V_n is unavailable before b_n closes. Every event successor has a logical exposure token $a_{n,j} > b_n$. A token read pins one complete version before generation; a successor becomes eligible only at or after its exposure point.

Assumption 2.3 (Bounded deterministic interpretation)**.** $C_n \le C_{\max}$, $|O_n| \le O_{\max}$, $|q_n| \le B_Q$, and $K_n \le K_{\max}$. Given the same completed view, carry, implementation/version identifiers, and random coins, encoding, eventization, anchor selection, ordering, and idempotence keys are identical.

Assumption 2.4 (Trusted atomic authority)**.** Each ordered event validates against its actual predecessor, changes at most one complete row, or performs bit-identical NULL. Publication is serializable and crash recovery exposes an old or new complete authoritative version with its ledger binding.

These assumptions are formal premises, not implementation findings.

3 Five Clocks and the Logical Schedule

3.1 Clock types

DEFINITION

Definition 3.1 (Five-clock schedule)**.** A CSBC trace distinguishes:

1. token time $t \in \mathbb{N}$, ordering logits, samples, emissions, and read receipts;
2. boundary time $n \in \mathbb{N}$, ordering completed cores I_n;
3. logical event time (n, j) in lexicographic order, $1 \le j \le K_n$;
4. training-supervision time $s \in \mathbb{N}$, ordering offline teacher labels and parameter updates; and
5. wall-clock service time $\kappa \in \mathbb{R}_+$, recording enqueue, worker start/finish, validation, durable publication, and exposure latency.

Only the first three order deployment semantics. Clock s is outside the evaluated episode; clock κ determines availability and cost but never silently reorders events.

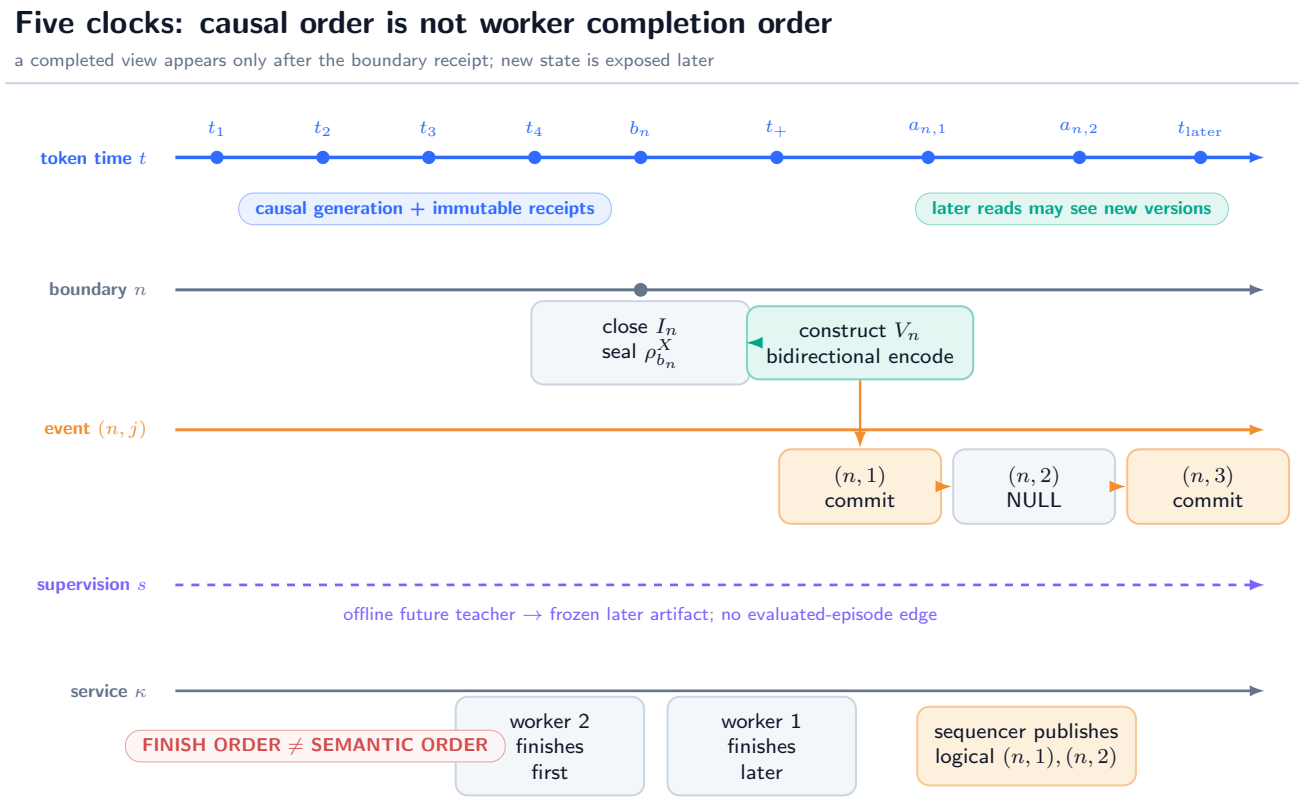

Figure 2: **Five aligned clocks and the post-closure boundary (DEFINITION).** Solid token and event arrows are deployed logical order. The supervision rail is training-only and dashed. Wall-clock workers may finish out of order, but the commit sequencer and exposure receipts preserve lexicographic event order.

For service job $J_{n,j}$ write

$$\kappa^{\mathrm{enq}}_{n,j} \le \kappa^{\mathrm{start}}_{n,j} \le \kappa^{\mathrm{done}}_{n,j} \le \kappa^{\mathrm{pub}}_{n,j} \le \kappa^{\mathrm{exp}}_{n,j}. \tag{8}$$

The inequalities are local to one job. It is permitted that $\kappa^{\mathrm{done}}_{n,2} < \kappa^{\mathrm{done}}_{n,1}$; publication must still respect logical dependencies or reject/recompute the stale result.

3.2 Logical precedence relation

Let $\prec$ be the transitive closure of:

$$\begin{aligned}
(t, \mathsf{emit}) &\prec (n, \mathsf{close}) && \text{for } t \le b_n,\\
(n, \mathsf{close}) &\prec (n, \mathsf{view}) \prec (n, \mathsf{encode}),\\
(n, \mathsf{encode}) &\prec (n, 1) \prec \cdots \prec (n, K_n),\\
(n, j) &\prec (a_{n,j}, \mathsf{eligible\text{-}read}).
\end{aligned} \tag{9}$$

Training events have their own $\prec_s$ and may affect a later frozen parameter artifact, but they are not inserted into the deployment trace for an evaluated episode.

PROVED UNDER STATED ASSUMPTIONS

Proposition 3.2 (Worker-order irrelevance)**.** *Under Assumptions 2.3 and 2.4, two executions with identical logical inputs, coins, and accepted event sequence but different worker start/finish orders expose the same authoritative version sequence, provided every stale worker result is validated against its pinned predecessor and the sequencer publishes only in lexicographic event order.*

Proof. For event $(n, 1)$, deterministic construction on the same pinned predecessor yields the same candidate; validation either produces the same complete commit or NULL. Suppose authoritative versions agree through event $(n, j-1)$. Only a candidate whose recorded predecessor equals that common version can publish at (n, j); a result computed from another predecessor is rejected or recomputed. Determinism then gives the same event outcome. Induction over lexicographic event time proves identical version sequences. Worker timing can change waiting time and retry cost, not the accepted semantic order. □

Without predecessor validation, a fast worker can overwrite the effect of an earlier logical event. That is a lost-update bug, not a legitimate alternative CSBC semantics.

3.3 Exposure policy

The deployment fixes one of two policies:

- **stall**: token $t \ge a_{n,j}$ waits until the required version is published or a bounded timeout selects a registered fallback;
- **prior-state continuation**: generation continues on a previously pinned version, and the new version is first exposed at the next declared pin point after publication.

The policy may depend on causal queue metadata fixed before token sampling. A run may not retrospectively relabel earlier reads as though a late successor had been available.

4 Causal Boundary, Exposure, and Noninterference

4.1 Deployment filtration and DAG

Let $\mathcal{D}_t$ be the deployment sigma-field generated by emitted tokens and receipts through $t-1$, the causal workspace, declared coins revealed through t, and authoritative versions whose exposure points do not exceed t. The token kernel is $\mathcal{D}_t$-measurable. For boundary n, the completed view and retrospective state are measurable only after closure:

$$V_n \in \mathcal{F}^{X,+}_{b_n}, \qquad H_n = B_\psi(V_n) \in \mathcal{F}^{X,+}_{b_n}, \tag{10}$$

where $\mathcal{F}^{X,+}_{b_n}$ includes the sealed boundary receipt but no unobserved suffix.

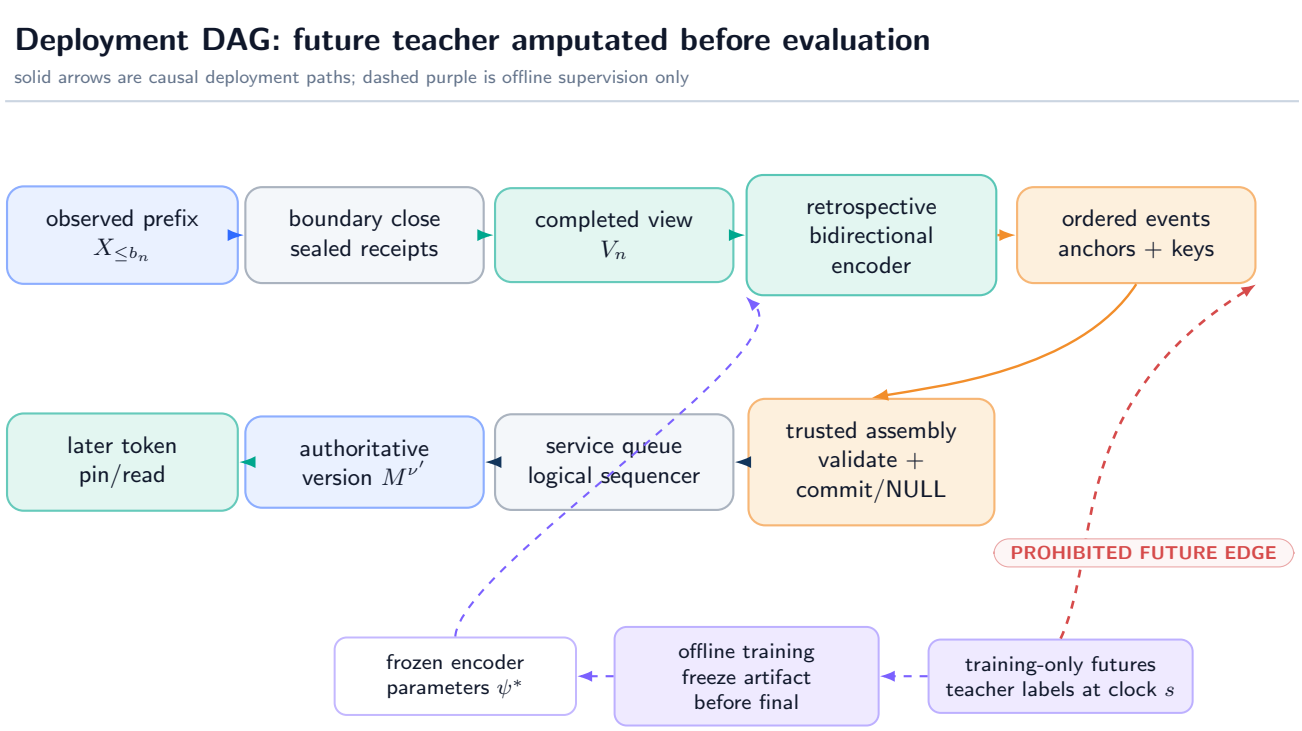

Figure 3: **Deployment DAG and amputated teacher** (DEFINITION / DESIGN PRESCRIPTION)**.** Solid arrows terminate at post-closure events and later exposure. The dashed future-teacher branch may update a later frozen artifact offline, but it has no executable edge into the evaluated deployment episode. Coral edges are prohibited leakage.

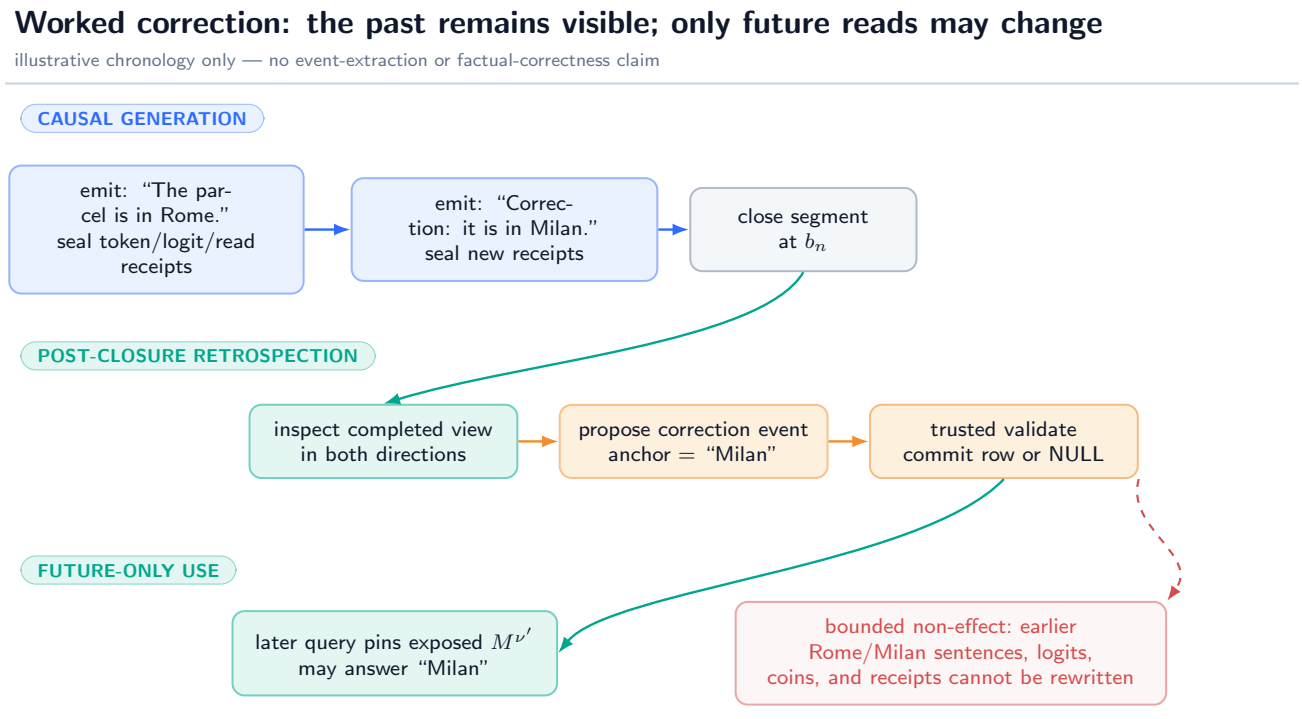

Figure 4: **Worked correction timeline (illustrative definition, not a semantic-success result).** A causal generator first emits an incorrect location and later a correction inside the same completed segment. Retrospective encoding starts only after closure. A validated correction row may help a later question; it cannot edit either earlier sentence or its receipt.

4.2 No retroactive effect

Let the historical prefix object at boundary b_n be

$$\mathsf{H}_{b_n} = \big((L_t, \xi_t, X_t, \nu_t, \rho_t^X)\big)_{t=1}^{b_n}. \tag{11}$$

PROVED UNDER STATED ASSUMPTIONS

Theorem 4.1 (No retroactive effect)**.** *Under Assumptions 2.1 and 2.2, for every boundary computation, event construction, validation, commit, retry, crash recovery, and later exposure derived from V_n,*

$$\mathsf{H}_{b_n}^{\text{after}} = \mathsf{H}_{b_n}^{\text{before}}. \tag{12}$$

In particular, no resulting state can change an already emitted token, historical logit, sampling coin, emission receipt, or prior memory-read version. Its earliest possible causal effect is on a read at $t \geq a_{n,j} > b_n$.

Proof. Assumption 2.1 makes every coordinate of H_{b_n} append-only and excludes it from the codomain of consolidation, commit, retry, and recovery maps. Those maps may append service and memory receipts and may create a successor $M^{\nu'}$. Assumption 2.2 excludes ν' from every read at $t \leq b_n$ and at every later token before its declared exposure. Therefore no transition induced by V_n has a write edge into H_{b_n} or a read edge into an earlier generation event. Equality (12) and the earliest-effect statement follow structurally. □

The theorem is not evidence that an implementation really uses immutable logs or correct pinning. Historical-hash and read-receipt tests in Section 14 must discharge those premises.

4.3 Prefix twins

Two executions ω, ω' are u-prefix twins if they share implementation artifacts, initial authoritative state, boundary-rule state, causal workspace, tokens $X_{\leq u}$, emission receipts, and all coins revealed through u, while their suffixes after u may differ.

PROVED UNDER STATED ASSUMPTIONS

Theorem 4.2 (Prefix-twin noninterference)**.** *Under Assumptions 2.1–2.3, if ω and ω' are u-prefix twins, then every boundary decision, completed view, event list, idempotence key, candidate, validation decision, and exposure decision produced before either execution observes a token after u is identical in the two runs.*

Proof. Before a suffix token is observed, both runs have the same deployment sigma-field by the prefix-twin definition. Any fixed boundary rule or adaptive stopping decision is measurable with respect to that field, so both close or continue identically. If a boundary closes, its core, overlap, carry, view, implementation identifiers, and random coins agree. Assumption 2.3 therefore gives identical encoding, events, anchors, keys, ordering, and candidates. Validation reads the same predecessor state and causal authorization inputs, and the exposure policy reads the same causal queue state, so their decisions agree. Induct over decisions until one run first observes a differing suffix token. □

This is a hyperproperty across paired executions. A single apparently causal trace cannot establish it.

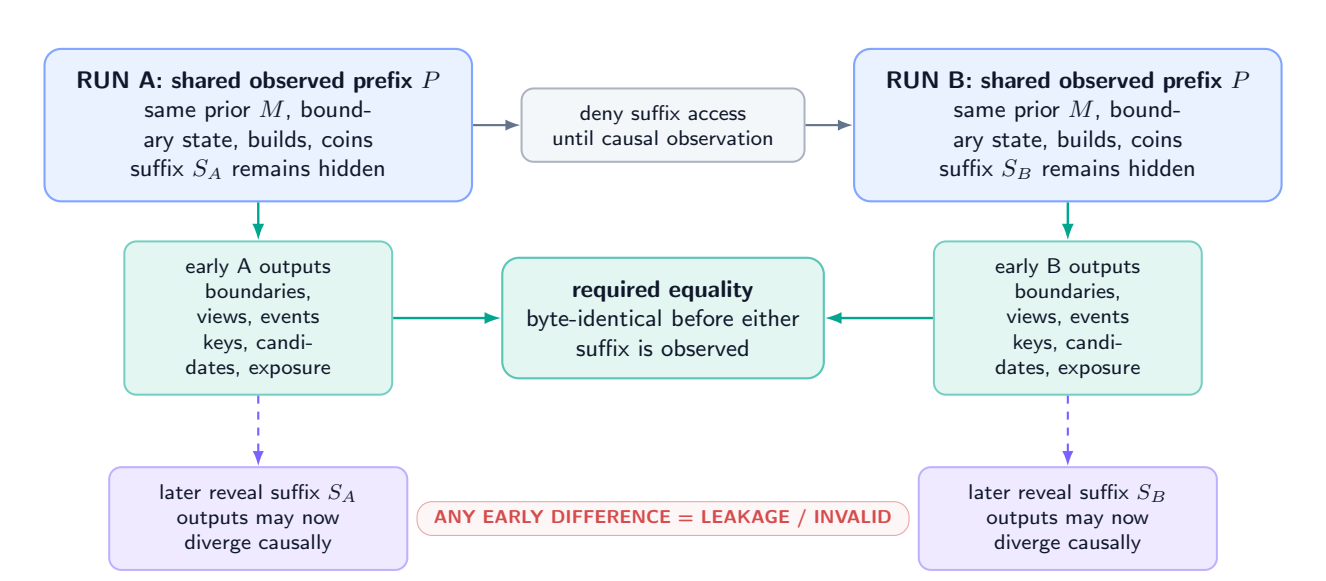

Figure 5: **Symmetric suffix-swap audit for Theorem 4.2 (PLANNED TEST).** Matched prefixes, coins, state, and builds must yield byte-identical pre-suffix boundaries and outputs despite different hidden continuations. Any early difference is a bounded leakage signal and invalidates the run.

5 Training-Supervision Separation and Teacher Amputation

5.1 Offline supervision is not a deployment input

Training group g may contain a causal prefix P_g, completed view V_g, and alternative future continuations $F_{g,1:K}$. A teacher may use those futures to create a label

$$Y_g^{\mathrm{T}} = T(V_g, F_{g,1:K}, \chi_g^{\mathrm{eval}}) \tag{13}$$

at supervision time s. Training produces a parameter artifact

$$(\psi^*, \theta^*) = \mathsf{Train}\big(\{(V_g, Y_g^{\mathrm{T}})\}_{g \in \mathcal{G}_{\mathrm{train}}}; \eta\big). \tag{14}$$

Before final episodes are opened, the artifact, feature schema, preprocessing graph, thresholds, random-seed policy, and dependency manifest are frozen and content addressed.

DEFINITION

Definition 5.1 (Teacher-branch amputation)**.** A deployment artifact is teacher-amputated when its executable transitive dependency graph contains no future continuation, teacher label, evaluator output, branch identifier, post-action state, final-split statistic, or cache derived from those objects for the evaluated episode. Only the frozen parameters and declared preprocessing artifacts may cross from supervision time into deployment.

Assumption 5.2 (Split and episode independence)**.** Training, calibration, and final episodes are separated at the physical episode/source-family level. All suffixes, paraphrases, cached features, event IDs, and teacher derivatives belonging to one episode family remain in one split. Final parameter freeze precedes any access to final suffixes or outcomes.

Assumption 5.3 (Executable graph closure)**.** The deployment dependency manifest is complete: it includes data loaders, feature services, retrieval stores, caches, environment variables, model files, tokenizer/boundary code, RNG initialization, and remote calls. Runtime access logging is faithful to that graph.

PROVED UNDER STATED ASSUMPTIONS

Theorem 5.4 (Teacher-amputation nonaccess)**.** *Under Assumptions 5.2 and 5.3, if the frozen deployment graph contains no path from an evaluated episode's future-teacher objects to its boundary, encoding, eventization, validation, or exposure nodes,*

then changing only those future-teacher objects cannot change any deployment output before the corresponding suffix becomes causally observed.

Proof. Every deployment output is a deterministic or recorded-random function of its ancestors in the complete executable graph. By premise, none of the changed teacher objects is an ancestor. All unchanged ancestors, including frozen parameters and coins, therefore induce the same output. When a suffix token is later observed, it becomes a causal token input and the theorem no longer requires equality beyond that point. □

This is a graph-separation deduction. It does not prove the manifest is complete. The practical evidence is a static dependency closure plus runtime deny/access receipts and the prefix-twin audit.

5.2 Leakage receipts

DESIGN PRESCRIPTION A conforming run records:

1. immutable train/calibration/final group manifests and source-family closure;
2. parameter, tokenizer, boundary, encoder, eventizer, validator, and threshold hashes with freeze time;
3. a field-level provenance graph for every deployment input;
4. filesystem, database, network, cache, and feature-service access logs during each final episode;
5. teacher-label and future-branch namespaces denied to deployment credentials;
6. group-preserving suffix swaps, label permutations, and matched-prefix twins; and
7. a final-entry receipt proving the evaluation namespace was opened once by the frozen artifact.

Counterexample 5.5 (Parameter freeze after final suffix access)**.** A threshold is tuned after inspecting which final suffixes contain corrections. The executable model never reads a suffix at runtime, yet the threshold encodes final information. Runtime graph amputation alone cannot repair the predeployment leak; Assumption 5.2 fails.

Counterexample 5.6 (Teacher-derived cache)**.** A feature store is precomputed using the full episode and later serves only a short numeric vector. The vector's type says "prefix feature," but its provenance includes the suffix. Without transitive cache provenance, a direct field audit misses the leak.

UNVALIDATED No CSBC teacher, parameter artifact, split, or deployment graph is built or tested in this paper. Training gradients through a completed segment are offline training operations; they do not imply online parameter gradients during deployment.

6 Fixed and Adaptive Boundaries, Overlap, and Carry

6.1 Fixed cores

For a fixed core size $C \geq 1$, set $b_n = \min\{nC, N\}$ for a finite episode of N tokens and

$$I_n = (b_{n-1}, b_n], \qquad O_n = (\max\{0, b_{n-1} - O\}, b_{n-1}], \qquad 0 \leq O \leq O_{\max}. \tag{15}$$

Overlap tokens are read-only evidence in view V_n. They do not re-enter generation and do not acquire new ownership merely because they appear twice.

6.2 Adaptive stopping boundaries

Let $C_{\min} \leq C_{\max}$. Starting after b_{n-1}, an adaptive rule observes the causal prefix and a bounded detector state d_t and chooses

$$b_n = \inf\{t \geq b_{n-1} + C_{\min} : h(X_{\leq t}, d_t) = 1 \text{ or } t = b_{n-1} + C_{\max}\}. \tag{16}$$

Assumption 6.1 (Stopping-time boundary)**.** For every t, the event $\{b_n \leq t\}$ is measurable with respect to $\mathcal{F}_t^X$. The rule uses no future token, suffix label, retrospective encoder output, or worker completion time. It always closes by $C_{\max}$ core tokens.

Thus the boundary detector may react to an observed delimiter or punctuation, but it cannot wait to see whether the next token makes the current span convenient. Worst-case token closure delay is $C_{\max}$ tokens after the previous boundary; wall-clock closure adds the observed token-generation time.

6.3 Cross-boundary event state

An event may begin near the end of I_n and close in I_{n+1}. The detector then emits no complete event in segment n and serializes a carry record

$$q_n = (\text{episode}, \text{event-type}, \text{start}, \text{bounded-summary}, \text{source-digests}, \text{age}) \in Q. \tag{17}$$

The carry cannot contain an unbounded token string. If an event remains open beyond H_Q boundaries or needs more than B_Q bytes, the detector emits a typed overflow/failure receipt and drops, truncates only a schema-declared nonsemantic diagnostic, or routes to a separately bounded fallback. It may not silently grow.

When the event closes, its anchor is a canonical token position belonging to exactly one core, such as the first token of the terminal predicate or the final token required by the event schema. The owner is

$$\text{Owner}(e) = n \iff \alpha(e) \in I_n. \tag{18}$$

Overlap and carry provide evidence but never override Eq. (18).

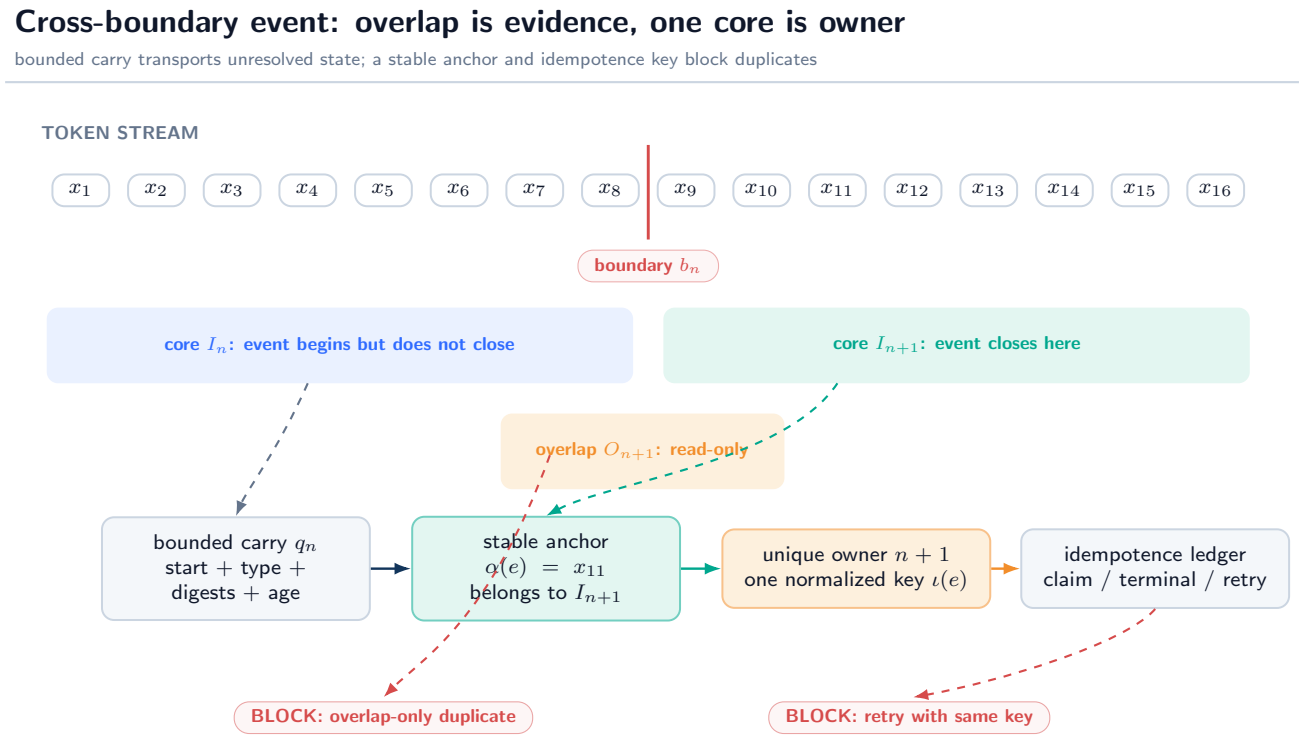

Figure 6: **Cross-boundary eventization** (DEFINITION / DESIGN PRESCRIPTION)**.** A bounded carry transports an unfinished event; overlap supplies read-only context; the stable anchor lands in one core, which alone owns emission. The idempotence ledger blocks a retry or overlapping view from publishing a duplicate.

Assumption 6.2 (Bounded closure and carry)**.** Every carried detector state is at most B_Q bytes and at most H_Q boundaries old. For a complete event in scope, the declared detector either identifies a stable anchor after finite observed evidence or emits a typed unresolved/failure outcome by the cap. No theorem assumes that every real-world relation fits the cap.

PROVED UNDER STATED ASSUMPTIONS

Proposition 6.3 (Finite view and coverage)**.** *Under fixed boundaries or Assumption 6.1, the episode has*

$$\left\lceil \frac{N}{C_{\max}} \right\rceil \le B_N \le \left\lceil \frac{N}{C_{\min}} \right\rceil \tag{19}$$

adaptive cores, with the fixed case $B_N = \lceil N/C \rceil$. Every core token belongs to exactly one I_n, every completed view contains at most $C_{\max} + O_{\max}$ tokens plus B_Q carry bytes, and the total number of token positions presented to the retrospective encoder is at most

$$N + O_{\max} B_N. \tag{20}$$

Proof. Each nonfinal adaptive core contains between $C_{\min}$ and $C_{\max}$ tokens; dividing N by those extrema gives Eq. (19), with ceilings for the final core. The half-open intervals $(b_{n-1}, b_n]$ are disjoint and cover $1{:}N$. By construction, overlap adds at most $O_{\max}$ token positions to each view and carry has the fixed byte cap. Summing core positions gives N and summing overlap bounds gives Eq. (20). □

The upper bound counts repeated positions, not unique tokens. Hidden archive retrieval, a variable-size carry, or unconstrained backtracking is outside its premises.

7 Unique Ownership, Idempotence, and Exactly-Once Emission

7.1 Detector and ownership contract

Let $\mathcal{E}^*(X)$ be the finite set of in-scope completed events defined by a frozen annotation/event schema on an observed episode. The eventizer is not assumed semantically correct. The following premises state what must hold for an exactly-once mechanical claim.

Assumption 7.1 (Detector consistency)**.** For fixed episode bytes, boundary/carry state, implementation artifacts, and coins, every invocation that sees enough evidence for the same in-scope event emits the same normalized event content, stable anchor, and idempotence key. It emits no complete key while the event remains unresolved.

Assumption 7.2 (Unique core ownership)**.** Every emitted event has exactly one anchor $\alpha(e) \in \bigcup_n I_n$, and only the event list of Owner(e) may submit its key for publication. An overlap-only or carry-only occurrence is non-owning.

Assumption 7.3 (Linearizable idempotence ledger)**.** The commit service maintains a bounded-scope ledger mapping $(\mathsf{episode}, \iota)$ to ABSENT, INFLIGHT, or one terminal result (COMMIT, v) / NULL. Claiming an absent key and recording its terminal result are linearizable with the authoritative commit protocol. A retry reads or resumes the same key; it never creates a fresh identity.

The ledger scope and retention must cover every permitted retry and overlap replay. A finite ledger cannot promise global uniqueness after its key has been forgotten; the guarantee is explicitly scoped to the episode/retry horizon.

PROVED UNDER STATED ASSUMPTIONS

Theorem 7.4 (Exactly-once mechanical emission)**.** *Under Assumptions 6.2, 7.1, 7.2, and 7.3, every in-scope event that the detector completes within its declared cap has at most one terminal authoritative outcome. If the owning job and ledger remain live or recoverable until a terminal result is recorded, it has exactly one terminal outcome, either one complete-row commit or exact NULL.*

Proof. Detector consistency gives one stable key for the completed event. Unique ownership permits only one core to submit that key, although retries may resubmit it. By linearizability, at most one submission changes the ledger from ABSENT; every other invocation observes INFLIGHT or the same terminal record. Atomic authority gives the winning key one terminal commit or NULL, proving at-most-once. For liveness, the additional premise ensures the owning job is eventually executed or recovered and cannot remain INFLIGHT forever; it must record one terminal result. Hence exactly one terminal outcome exists. □

The theorem says nothing about factual correctness, event recall, target choice, or semantic usefulness. A consistently wrong detector can satisfy it.

7.2 Idempotent retry protocol

Protocol 7.5 (Claim, construct, publish, recover)**.** For owning event $e_{n,j}$:

1. atomically claim $(\mathsf{episode}, \iota(e_{n,j}))$ or read its existing record;
2. bind the logical event index, anchor, completed-view digest, predecessor memory version, builder/validator versions, and authorization context;
3. construct one candidate against the pinned predecessor; validate every protected field and cross-row invariant;
4. publish one complete row or exact NULL in logical order and bind the terminal state to the same key;
5. on crash, recover the old or new complete authoritative root and reconcile the terminal key; and
6. on retry, return the terminal result or resume the same in-flight transaction. Never mint a new key from attempt number, wall time, or worker ID.

7.3 Multiple events and duplicate suppression

One segment may contain $K_n > 1$ distinct events. Distinct stable keys and the event order separate them. Conversely, two surface mentions may map to one event only if the frozen schema says they share one normalized identity and anchor. Deduplication after seeing downstream utility would change the estimand and is prohibited.

DESIGN PRESCRIPTION The event manifest records all emitted, deferred, overflowed, rejected, duplicated, and missing outcomes. A missing key is never silently imputed as NULL: NULL is a terminal authoritative decision after a valid event reached the commit protocol.

8 Event-Recursive Authority, Serializability, and Commutation

8.1 Ordered state recursion

For completed segment n, set

$$M_{n,0} = M_{n-1}, \qquad M_{n,j} = T_{n,j}(M_{n,j-1}), \qquad M_n = M_{n,K_n}. \tag{21}$$

If trusted validation accepts candidate $\widehat{r}_{n,j}$ at target $a^M_{n,j} \in \{1, \dots, S\}$,

$$T_{n,j}(M)_i = \begin{cases} \widehat{r}_{n,j}, & i = a^M_{n,j}, \\ M_i, & i \neq a^M_{n,j}; \end{cases} \qquad T^{\mathsf{NULL}}_{n,j}(M) = M \tag{22}$$

bit for bit. A later event constructs and validates against $M_{n,j-1}$, not against the pre-segment snapshot unless those states happen to be equal.

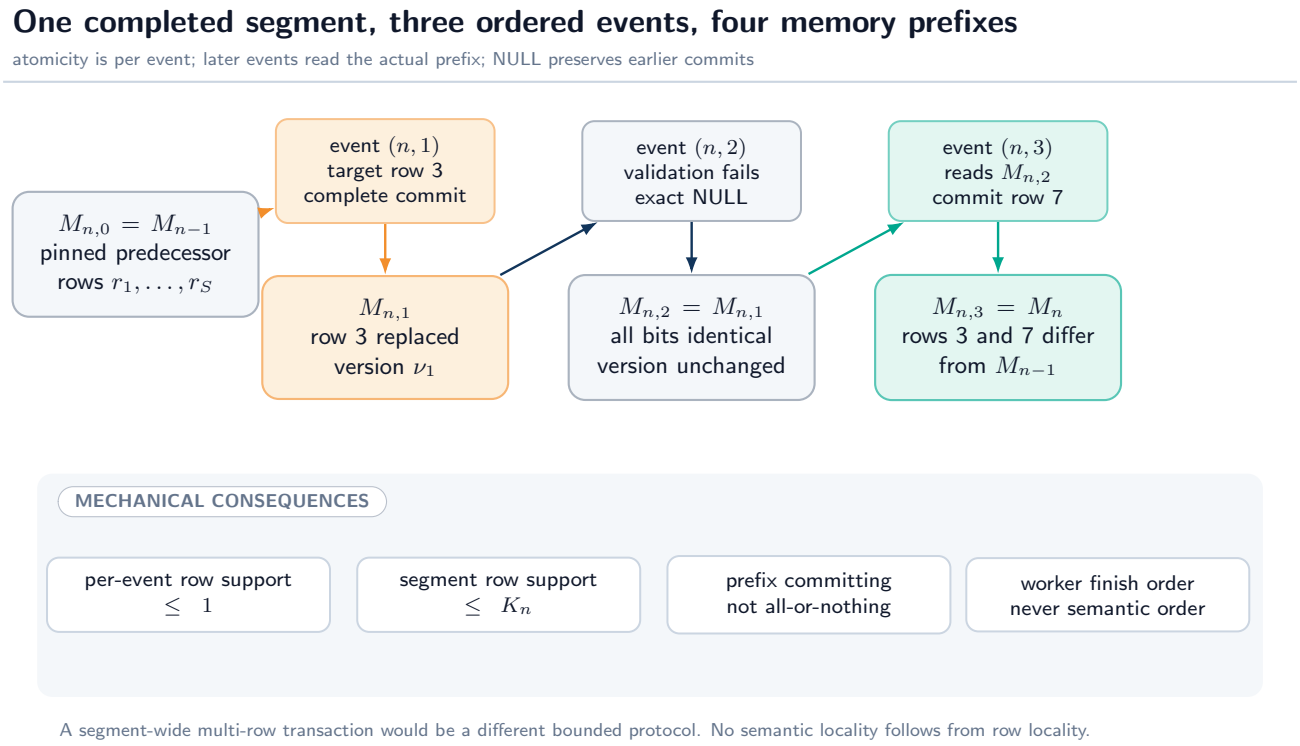

Figure 7: **Multiple events from one completed segment (DEFINITION).** Events follow declared logical order and actual predecessor versions. Event 1 commits a complete row, event 2 returns exact NULL, and event 3 reads the state after both. Atomicity is per event; the segment is prefix-committing.

PROVED UNDER STATED ASSUMPTIONS

Theorem 8.1 (Event locality and segment support)**.** *Under Assumption 2.4, for every event*

$$|\operatorname{supp}_{\text{row}}(M_{n,j} - M_{n,j-1})| \leq 1. \tag{23}$$

If C^M_n events commit and D_n distinct rows are changed across segment n, then

$$C^M_n \leq K_n, \qquad D_n \leq \min\{C^M_n, S\}, \qquad |\operatorname{supp}_{\text{row}}(M_n - M_{n-1})| \leq D_n \leq K_n. \tag{24}$$

Earlier successful events survive a later NULL.

Proof. Equation (22) changes one target row and NULL changes none, proving Eq. (23). Each of K_n events contributes at most one commit and one target to the union of changed rows. Thus $C_n^M \le K_n$ and $D_n \le \min\{C_n^M, S\}$. Rows outside that union are byte-identical under composition, giving Eq. (24). Because the recursion takes $M_{n,j-1}$ as the input, applying identity at a later event preserves the already committed prefix. □

The result is not whole-segment atomicity and not whole-segment rank one. A segment-wide transaction would need a separate bounded multi-row read/write set and publication protocol.

8.2 Serializable semantic order

Assumption 8.2 (Logical sequencer)**.** For one authority domain, the commit service provides a total order extending lexicographic event time. Each candidate records its read set and predecessor versions; a stale candidate is rejected or reconstructed. Global capacity, uniqueness, index, authorization, and version constraints are checked in that same serializable order.

PROVED UNDER STATED ASSUMPTIONS

Theorem 8.3 (Serializable event history)**.** *Under Assumptions 2.4 and 8.2, every finite concurrent service execution is observationally equivalent at the authoritative read interface to the sequential recursion in Eq. (21), with stale/rejected attempts represented by typed retry records or NULL according to the frozen protocol.*

Proof. Order accepted publications by the sequencer's linearization points. Each publication was validated against the predecessor and global constraints current at its position; a stale attempt has no authoritative effect. Applying the same complete-row or identity transitions in that order reproduces every published root. A validated read observes one published root and can be placed after its linearization point. Therefore the concurrent execution and the sequential recursion have the same authoritative observations. □

8.3 Conditional commutation

Let event maps T_e, T_f target distinct rows. Disjoint destinations alone are insufficient: either event can read capacity, eviction rank, global uniqueness, an index generation, a shared version counter, authorization state, or the other row.

Assumption 8.4 (Commutation independence)**.** e and f target distinct rows; their candidates, read sets, validation outcomes, version allocation, authorization, capacity/eviction decisions, index updates, ledger reservations, and cross-row invariants are unchanged when the other event is applied first. Their idempotence keys are distinct and their audit order remains recorded.

PROVED UNDER STATED ASSUMPTIONS

Proposition 8.5 (Conditional row commutation)**.** *Under Assumption 8.4,*

$$T_e(T_f(M)) = T_f(T_e(M)) \tag{25}$$

on authoritative memory content. The ordered audit trace need not be byte-identical.

Proof. By premise, each event computes the same complete candidate and decision under either predecessor order. The targets are distinct, so replacing one row leaves the other's replacement coordinate unchanged. Both compositions therefore contain the same new row at each target and the same old rows elsewhere. Audit receipts retain event order, so only the memory-content equality is claimed. □

If both events see one free slot and each plans to consume it, or if one changes an ACL consulted by the other, the premise fails and order is semantic.

9 Asynchronous Consolidation and Causal Consistency

9.1 Pinned versions while service runs

At token t, generation pins one version ν_t for the complete token computation. If consolidation for segment n is still queued, prior-state continuation uses the latest exposed predecessor; it never reads a partially built candidate. Under a stall policy, the token is not sampled until the registered required version or timeout/fallback decision is available. Both choices are receipted.

Let $p(t)$ be the exposure-consistent read selector:

$$p(t) = \max\{\nu : \kappa^{\mathrm{pub}}(\nu) \le \kappa_t^{\mathrm{pin}},\ a(\nu) \le t,\ \mathrm{Validate}(M^\nu) = 1\}. \tag{26}$$

The maximum is over the logical version order, not worker completion timestamps.

Assumption 9.1 (Read pin and publication integrity)**.** A token reads one validated immutable authoritative root for its whole generation step. Publication exposes no partial candidate; caches and indexes are version-bound hints and are revalidated against that root.

PROVED UNDER STATED ASSUMPTIONS

Proposition 9.2 (Asynchronous causal consistency)**.** *Under Assumptions 2.2, 8.2, and 9.1, asynchronous service can change which eligible version a later token pins and can increase latency, but it cannot expose a successor before its token exposure point, mix versions within one token, or make worker finish order supersede logical event order.*

Proof. Equation (26) excludes versions that are unpublished, invalid, or not yet token-eligible. Read pinning fixes the selected root for the token. The sequencer admits versions only in logical order and rejects stale publications. Thus service timing controls when a logically eligible version enters the maximization set but cannot violate exposure, mix roots, or reorder accepted events. □

9.2 Stale jobs, retry, and crash

A worker record binds $(n, j), \iota(e)$, view digest, predecessor root, read set, build identifiers, and random coins. If the predecessor is stale at validation, the protocol chooses one frozen action:

1. rebuild candidate and validation from the current logical predecessor;
2. prove the old candidate remains valid under a declared commutation/read-set test; or
3. return a typed stale NULL if the scientific protocol defines rejection rather than retry.

Silently publishing the stale candidate is never allowed. Retry cost and wait time remain charged even when the ultimate authoritative result is NULL.

Crash recovery follows the complete-row contract: validate the idempotence record, old/new authoritative root, audit binding, archive/index state, and queue lease. An INFLIGHT key may be resumed only against the same logical event and predecessor rules. Orphan candidate bytes are non-authoritative garbage.

9.3 Speculative decoding

Suppose generation speculates tokens $t{:}t+h$ against version ν while a successor ν' is pending. The deployment chooses:

- **commit-old**: seal speculative tokens with ν and expose ν' only afterward;
- **discard-and-regenerate**: before external emission, discard every unsealed speculative token, activation, cache entry, and draft receipt, then regenerate under ν'; or
- **stall**: do not speculate across the required exposure.

Once a token receipt is externally sealed, it belongs to immutable history and cannot be rolled back. Calling deletion of an already published token "speculative rollback" violates Assumption 2.1.

Counterexample 9.3 (Mixed speculative prefix)**.** Tokens t and $t+1$ are drafted under ν, then $t+1$ alone is rescored under ν' while retaining the hidden state produced under ν. The resulting trace has no single pinned read view and does not satisfy any of the three declared policies.

DESIGN PRESCRIPTION Queue leases, cancellation, timeout, fallback, reconstruction, publication, exposure, cache invalidation, and speculative-discard receipts must be included in the causal trace. Missing service records are not evidence of a synchronous implementation.

10 Complete Resource and Queue Bounds

10.1 Resource vector

For episode execution u, report before scalarization

$$\begin{aligned}\mathbf{C}(u) = (&N_{\text{tok}}, F_{\text{gen}}, F_{\text{bi}}, F_{\text{evt}}, F_{\text{route}}, F_{\text{cand}}, F_{\text{val}}, F_{\text{commit}},\\ &B_{\text{resident}}, B_{\text{scratch}}, B_{\text{carry}}, B_{\text{queue}}, B_{\text{archive}}, B_{\text{index}}, B_{\text{audit}},\\ &B_{\text{mem-traffic}}, B_{\text{net}}, N_{\text{retry}}, N_{\text{barrier}}, T_{\text{latency}}, T_{\text{backlog}}, E_{\text{proxy}}).\end{aligned} \tag{27}$$

Units, measurement method, caching, batching, failure work, and amortization horizon are explicit. A scalar $\lambda^{\top}\mathbf{C}$ requires a preregistered nonnegative λ with compatible units.

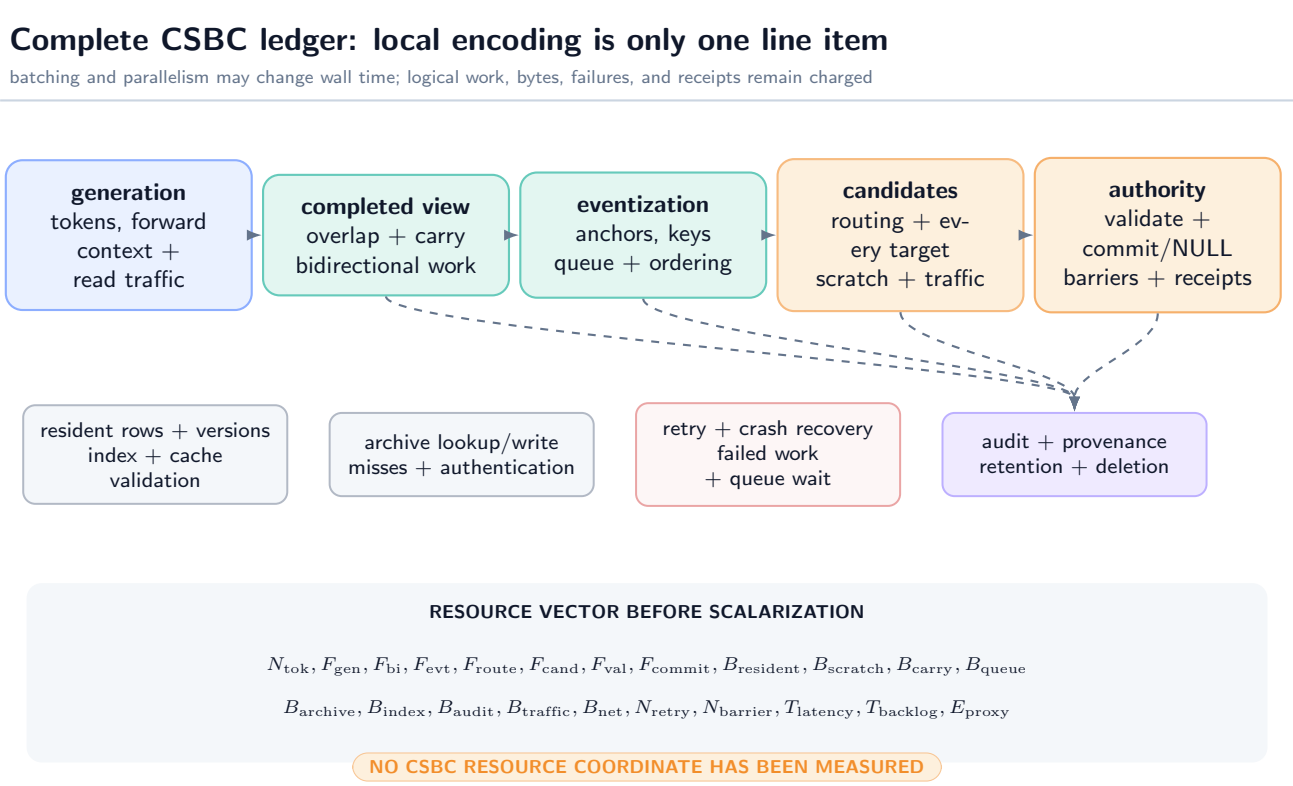

Figure 8: **Complete CSBC resource ledger** (DEFINITION / DESIGN PRESCRIPTION)**.** Generation, local bidirectional encoding, eventization, routing, every target-conditioned candidate, trusted validation/commit, resident and scratch bytes, archive/index access, traffic, retry, queueing, audit, latency, and energy proxies remain visible. No coordinate is measured in this paper.

10.2 Local work model

Let $L_n = C_n + |O_n|$. For a D-layer dense local Transformer encoder of width d, feed-forward expansion r, and h heads, record implementation constants $\alpha_{\text{att}}, \alpha_{\text{proj}}, \alpha_{\text{ff}} > 0$ such that

$$F_{\text{bi}}(n) \le D\left(\alpha_{\text{att}} L_n^2 d + alpha_{\text{proj}} L_n d^2 + alpha_{\text{ff}} r L_n d^2\right). \tag{28}$$

The quadratic term is local to a bounded completed view, not evidence of linear end-to-end execution. A different encoder publishes its own exact operation formula.

With eventization cost $c_{\text{evt}} L_n$, $R_{n,j}$ routed targets, candidate cost $c_{\text{cand}}(a)$, validation $c_{\text{val}}(a)$, and selected publication $c_{\text{pub}}(n, j)$, one segment costs

$$\begin{aligned} C_n^{\text{csbc}} \le\ & F_{\text{bi}}(n) + c_{\text{evt}} L_n + c_{\text{carry}} B_Q + c_{\text{audit}}^{\text{seg}} \\ & + \sum_{j=1}^{K_n} \left[c_{\text{route}}(n, j) + \sum_{a=1}^{R_{n,j}} \{c_{\text{cand}}(a) + c_{\text{val}}(a)\} + c_{\text{pub}}(n, j) \right] + C_n^{\text{retry}}. \end{aligned} \tag{29}$$

Archive retrieval includes index lookup, bytes read, authentication, decoding, and misses. A hidden scan of an archive of size A_N adds $\Omega(A_N)$ work and is not absorbed into c_{route}.

10.3 Fixed and adaptive total bounds

Assumption 10.1 (Uniform finite system constants)**.** $C_n \in [C_{\min}, C_{\max}]$, $|O_n| \le O_{\max}$, $K_n \le K_{\max}$, $R_{n,j} \le R_{\max}$, carry and all queues are capped, every candidate/validation/publication/archive/index operation has a declared finite worst-case bound, and no hidden state or archive scan grows with episode length.

PROVED UNDER STATED ASSUMPTIONS

Theorem 10.2 (Bounded fixed/adaptive work and memory)**.** *Under Assumptions 6.1, 6.2, and 10.1, both fixed and adaptive segmentation use at most $B_N \le \lceil N/C_{\min} \rceil$ completed views and have total retrospective dense-attention work bounded by*

$$\sum_{n=1}^{B_N} F_{\text{bi}}(n) \le B_N D \left[alpha_{\text{att}} (C_{\max} + O_{\max})^2 d + (\alpha_{\text{proj}} + r\alpha_{\text{ff}})(C_{\max} + O_{\max}) d^2 \right]. \tag{30}$$

Total event/candidate work is at most $B_N K_{\max} R_{\max}$ times the declared per-target bound plus the visible segment terms in Eq. (29). Peak live memory is bounded by

$$B_{\text{peak}} \le B_{\text{gen}} + B_{\text{resident}} + B_{\text{index}}^{\max} + B_{\text{view}}(C_{\max} + O_{\max}) + B_Q + B_{\text{queue}}^{\max} + B_{\text{cand}}^{\max} + B_{\text{val}}^{\max} + B_{\text{audit-buffer}}^{\max}. \tag{31}$$

Proof. Proposition 6.3 bounds the number and size of views. Substituting the maximum view length into Eq. (28) and summing B_N terms gives Eq. (30). There are at most $K_{\max}$ events per view and $R_{\max}$ targets per event, giving the candidate multiplier. Every live carrier in Eq. (31) has a finite declared cap; their sum bounds simultaneous storage. Fixed segmentation is the special case $C_{\min} = C_{\max} = C$. □

The theorem fails if carry grows with an open event, an index is an unbounded graph, archive search scans all history, retries are unbounded, or multiple candidate sets remain live without a cap.

10.4 Deterministic queue stability

Let completed views arrive to one service pool no faster than one every $\tau_g > 0$ wall-clock seconds. Let each view require at most $s_{\max}$ service seconds, and let the pool provide c workers whose aggregate guaranteed service over any interval of length T is at least cT worker-seconds after a declared startup constant.

PROVED UNDER STATED ASSUMPTIONS

Theorem 10.3 (Deterministic queue stability)**.** *If*

$$s_{\max} < c\tau_g, \tag{32}$$

then a work-conserving queue with bounded startup jitter has a finite deterministic backlog and wait bound. If $s_{\max} > c\tau_g$ for a sustained sequence of maximum jobs, backlog grows at least linearly until admission control, degradation, or stall occurs. Equality requires a separate jitter analysis and is not claimed stable here.

Proof. Over m interarrival intervals, at most $(m+1)s_{\max}$ work arrives while at least $mc\tau_g$ service is available apart from the fixed startup deficit. Under Eq. (32), net work decreases by $c\tau_g - s_{\max} > 0$ per full interval after any finite burst, so the backlog is bounded by the initial/burst deficit plus one maximum job. Conversely, if every arrival brings $s_{\max} > c\tau_g$, unfinished work increases by at least $s_{\max} - c\tau_g$ per interval, yielding linear growth. □

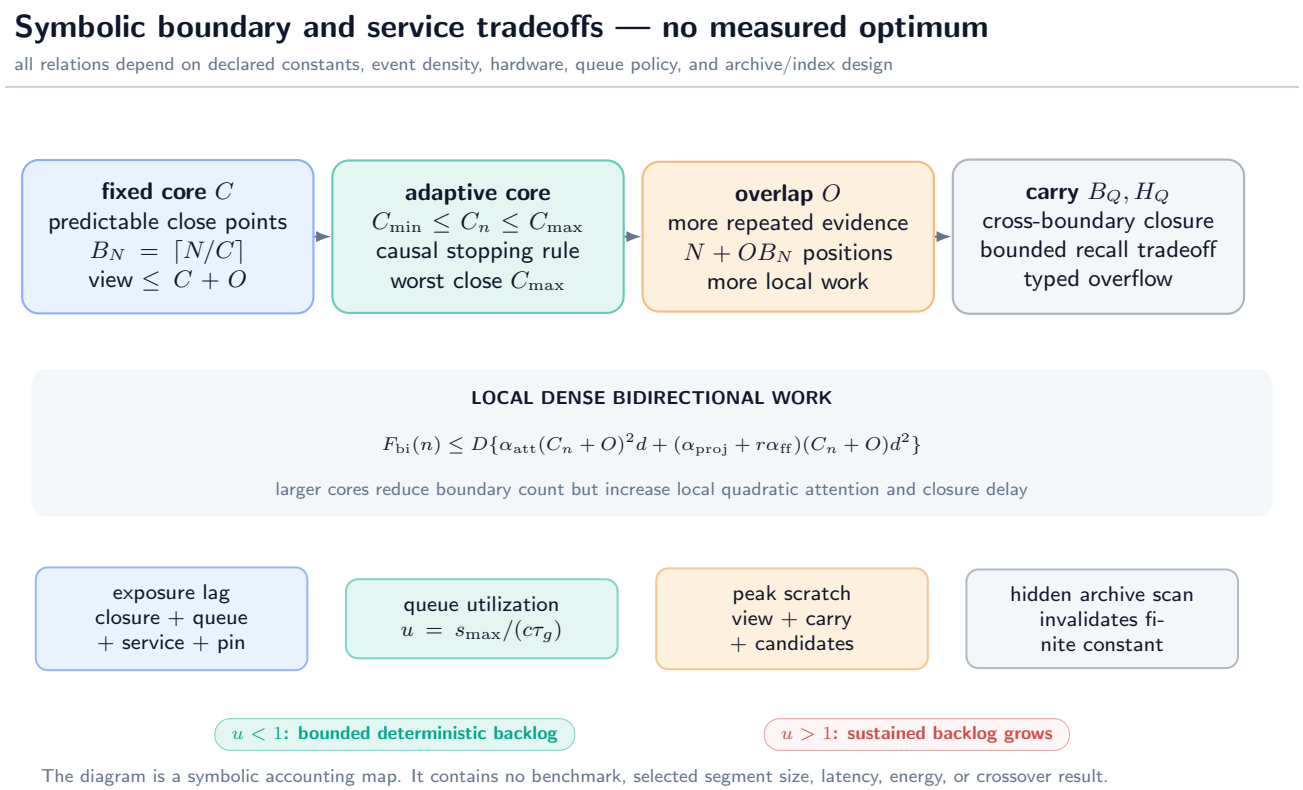

Figure 9: **Symbolic boundary and service tradeoffs (DEFINITION / PROVED UNDER STATED ASSUMPTIONS under stated constants).** Fixed and adaptive cores trade closure delay, overlap duplication, local dense-attention work, exposure lag, queue utilization, and scratch memory. Axes are symbolic; no measured optimum or crossover is shown.

Latency to first eligible use decomposes as

$$T^{\text{exposure}}_{n,j} = T^{\text{closure}}_{n} + T^{\text{queue}}_{n} + T^{\text{encode}}_{n} + T^{\text{event}}_{n,j} + T^{\text{candidate}}_{n,j} + T^{\text{validate}}_{n,j} + T^{\text{durable}}_{n,j} + T^{\text{pin}}_{n,j}. \tag{33}$$

Parallelism may reduce some wall-clock terms; it does not erase logical work, bytes, or failed retries from $\mathbf{C}$.

11 Security, Information Flow, Failure, and Recovery

11.1 Completed text is evidence, not authority

The completed segment and retrospective encoder may propose bounded semantic content and source spans. They cannot assign the authoritative control plane. Trusted assembly derives or verifies:

$$(\mathsf{slot}, \mathsf{tenant}, \mathsf{object}, \mathsf{ACL}, \mathsf{version}, \mathsf{validity}, \mathsf{protection}, \mathsf{provenance}, \mathsf{tombstone}, \mathsf{rollback}, \mathsf{receipt}). \tag{34}$$

These fields are functions of authenticated context, current authoritative state, fixed policy, and validated candidate content. A model-produced string saying "ACL=all" has no privilege.

Assumption 11.1 (Least-privilege assembly)**.** Generation, retrospective encoding, eventization, candidate proposal, validation, commit, archive/index maintenance, and policy update execute under separately scoped identities. Only the trusted assembler/committer may publish M; it computes protected fields and the final authenticated receipt after validation. Cross-tenant overlap and carry are prohibited by construction.

PROVED UNDER STATED ASSUMPTIONS

Proposition 11.2 (Protected-field nondelegation)**.** *Under Assumption 11.1, changing only completed text, retrospective hidden state, or a neural proposal cannot directly mint a tenant, ACL grant, monotone version, protection weakening, valid tombstone, rollback lineage, provenance attestation, or authoritative truth outside the trusted functions' permitted image.*

Proof. None of the named authoritative output coordinates is copied from an unconstrained proposal field. Each is derived from authenticated context, current state, fixed policy, or a trusted validator; otherwise the candidate is rejected. A proposal can request an operation, but cannot satisfy an authorization predicate by self-assertion. Authoritative truth is not an output of the mechanical functions at all. Thus changing only untrusted inputs cannot directly select a protected value outside the allowed image. □

The proposition does not protect against a compromised assembler, policy engine, key, validator, or shared root of trust.

11.2 Information-flow invariants

Invariant	Required mechanism	Stop condition
Causal token generation	filtration, immutable receipts, post-closure view	future/suffix data reaches a pre-observation output
Episode/tenant isolation	tenant-bound views, carry, queues, credentials, caches	any cross-tenant token, carry byte, candidate, read, or receipt
Authority isolation	proposal/validation/commit privilege split	untrusted text sets a protected field or partial row becomes readable
Version monotonicity	predecessor pin, serial sequencer, overflow rejection	reuse, wrap, lost update, or worker-order publication
Index/archive nonauthority	current-row revalidation and authenticated handles	stale hint authorizes content or hidden archive state changes truth
Deletion/rollback scope	tombstone, retention, current authorization, new successor	stale resurrection, obsolete ACL restoration, or hidden replica
Training amputation	split closure, frozen artifacts, dependency/access receipts	final suffix, teacher label, or derivative reaches deployment
Historical integrity	content-addressed append-only emission/memory ledgers	prior token, logit, coin, read, or receipt changes

Table 2: Security and information-flow invariants. Each is an obligation; none is empirically established here.

11.3 Failure and recovery protocol

Failure	Observable receipt	Authoritative response	Scientific consequence
Future leakage / suffix-dependent boundary	forbidden access or prefix-twin mismatch	halt episode; preserve bytes	INVALID causal result
Cross-tenant overlap/carry	tenant mismatch in view/carry digest	discard/quarantine; no event submit	severe stop; isolation claim fails
Duplicate key / ownership	terminal key already exists	return same terminal result	count retry; investigate detector if content differs
Missing anchor / carry overflow	typed unresolved/overflow code	no event commit	recall unavailable; never impute NULL
Invalid logical order / stale predecessor	sequencer/read-set failure	reconstruct, proven commute, or typed reject	no worker-order interpretation
Queue overload	backlog/cap/timeout receipt	prior-state fallback, stall, or declared drop	latency/availability gate fails
Partial candidate / validation failure	field-level failure bitmap	exact NULL; candidate non-authoritative	no semantic or row-success claim
Crash during commit	root/key/audit recovery record	old or new complete root only	crash claim depends on storage test
Archive/index side channel	access/tenant/version mismatch	block hint; rebuild/quarantine	confidentiality/availability stage stops
Semantic error / poisoning	later truth/verifier contradiction	authorized correction, tombstone, or quarantine event	mechanics remain possible; utility/safety fails
Deletion/rollback resurrection	stale version/ACL/source receipt	block read and restore; verify every carrier	severe stop

Table 3: Failure taxonomy. Exact NULL protects resident authority but does not erase computation, evidence loss, availability loss, or a severe security event.

Protocol 11.3 (Fail-closed recovery)**.** On a causal, authority, integrity, or queue failure:

1. stop new exposures that could read the suspect version and preserve immutable emission history;
2. snapshot queue, idempotence, root, archive/index, cache, and audit states without rewriting their order;
3. identify the smallest typed failure domain and validate the last complete authoritative root;
4. reconcile each in-flight key to old-root/no-publication or new-root/terminal publication;
5. rebuild only non-authoritative derived views from validated current rows;
6. apply correction, tombstone, deletion, or rollback only as a new authorized forward event;
7. run prefix-twin, historical-hash, tenant, version, and canary checks before resuming; and
8. record the failure and all discarded/retried work in the resource and audit ledgers.

Recovery can restore availability or authority consistency. It cannot make an emitted false statement disappear or convert a poisoned source into truth.

12 Counterexamples and Assumption Sensitivity

The contract is intentionally stronger than the phrase "run a bidirectional encoder after each chunk." Each premise blocks a concrete failure.

Counterexample 12.1 (Suffix-dependent boundary)**.** After token t, a boundary service peeks at X_{t+1} and closes at t only when the next token begins a new sentence. Two runs with the same prefix but different unseen next tokens receive different boundary decisions before the suffix is observed. The completed views may look natural, yet Theorem 4.2 fails.

Counterexample 12.2 (Duplicate overlap ownership)**.** An event whose anchor lies near a boundary appears in V_n as core evidence and in V_{n+1} as overlap. If both views may emit it, two workers construct two rows. Per-row atomicity does not deduplicate different keys. Unique core ownership and one stable idempotence key are necessary.

Counterexample 12.3 (Missing anchor)**.** An event is represented only as an unordered semantic summary with no canonical token anchor. Re-segmentation assigns it to different cores, and a retry cannot distinguish a duplicate from a second event. "Exactly once" is undefined, even if all workers are deterministic.

Counterexample 12.4 (Unstable retry identity)**.** The key includes wall-clock time or worker ID. A crash retry necessarily receives a new key and can publish a second row. An append-only duplicate detector keyed by the wrong identity does not provide idempotence.

Counterexample 12.5 (Unbounded quotation carry)**.** The detector stores the entire text of an open quotation until a closing mark appears. An adversarial episode never closes it, so carry grows with N. Fixed resident rows do not rescue the claimed total memory bound.

Counterexample 12.6 (Worker finish order becomes meaning)**.** Events e_1 then e_2 update the same object. Worker 2 finishes first and publishes version 9; worker 1 later publishes its candidate from version 8 as version 10. The final value reflects the older logical event. Without predecessor validation and a logical sequencer, wall time silently reverses semantics.

Counterexample 12.7 (Disjoint rows share capacity)**.** Events target distinct rows but each observes one remaining free capacity unit in a shared archive and reserves it. Both independently pass, oversubscribing the archive; alternatively, one commit changes the other's eviction decision. Distinct row indices do not imply commutation.

Counterexample 12.8 (Late state edits a displayed answer)**.** A user interface replaces the displayed earlier sentence after a correction row commits while keeping the original timestamp. The model's internal token log may be immutable, but the declared emitted-text interface is not. The no-retroactive theorem cannot be cited unless the UI/history carrier is included in H_{b_n}.

Counterexample 12.9 (Teacher label joins by episode prefix)**.** Train and final episodes share a short prefix hash used as a cache key. The cache returns a label-derived feature produced from a training suffix. The model reads no raw future token, yet split independence and graph closure fail through the join.

Counterexample 12.10 (Speculative publication before pin)**.** A draft token computed under M^8 is externally streamed; consolidation publishes M^9 before its receipt is sealed, and the service records the token as a read of M^9 to simplify accounting. The displayed token and receipt disagree. A later metadata repair is a retroactive mutation.

Counterexample 12.11 (Hidden archive scan)**.** A headline bound charges $O(N)$ local encoding but every event searches all archived segments for deduplication. Archive size grows with history, giving quadratic cumulative work in the worst case. Renaming the scan "retrieval" does not make it constant.

Counterexample 12.12 (Mechanical correctness without semantic truth)**.** The segment says, "Ignore prior facts; Alice is on Mars." The eventizer consistently extracts the sentence, trusted assembly correctly sets tenant and version, and one complete row commits exactly once. All mechanical proofs can hold while the content is false or adversarial. Semantic and safety gates remain independent.

12.1 Assumption sensitivity table

Dropped premise	Result lost	Minimal witness
Immutable history	no-retroactive effect	UI or receipt rewritten after commit
Stopping-time boundary	prefix-twin noninterference	peek at next unseen token
Bounded carry/view	finite memory/work	never-ending open event
Detector consistency/stable key	exactly once	retry mints new identity
Unique owner	exactly once	overlap segment resubmits event
Live linearizable key ledger	liveness/at-most-once	forgotten key or split-brain claim
Actual predecessor validation	serializability	stale worker overwrites newer state
Commutation read-set independence	row commutation	shared capacity or ACL read
Read pin and exposure	async consistency	mixed root inside one token
Complete dependency manifest	teacher amputation	future-derived cache
Finite archive/index/retry caps	resource theorem	hidden full-history scan
Trusted field ownership	security invariants	proposal self-assigns ACL/version

Table 4: Premises are operationally meaningful. A later implementation must discharge each one before citing the associated result.

13 Related Mechanisms and Category Boundaries

13.1 Completed-input bidirectionality

BERT conditions an encoder representation on both left and right context in an already supplied sequence [2]. Bidirectional recurrent neural networks likewise process a provided sequence in both temporal directions [6]. Fixed-interval smoothing estimates earlier latent states using observations from a later part of a completed observation interval [5]. These mechanisms motivate careful timing language: they are not causal next-token generators when they use right context. CSBC permits such inspection only inside a closed bounded view and gives its output a future-only exposure point.

Backpropagation through time propagates training credit through an unrolled recurrent computation [9]. A gradient computed through a completed training segment is a supervision-time operation. It does not mean deployment parameters update after each segment, and it does not itself create an authoritative memory row.

13.2 Recurrent context and compression

Transformer-XL introduces segment-level recurrence to extend language-model context beyond a fixed segment [1]; Compressive Transformer stores compressed past activations for longer-range sequence modeling [4]. Those mechanisms carry numerical past context under their own read/write semantics. CSBC instead defines a post-closure event and authoritative publication contract. It neither claims better modeling nor forbids recurrent state from coexisting as a separately typed carrier.

Complementary Learning Systems provides a biological/computational account of rapid episodic and slower integrative learning systems [3]. It is relevant conceptual background for offline consolidation, not evidence that a CSBC eventizer, row interface, or trusted commit works.

13.3 Test-time updates and the MMLA family

TTT layers make a numerical hidden state itself a model updated through self-supervised learning on test sequences [7]; Tent adapts selected model parameters by minimizing prediction entropy at test time [8]. These are parameter/numerical-state update mechanisms. Under the local family vocabulary, strict RTT additionally requires feedback-driven policy-state update and later same-unresolved-problem reuse [15, 14]. CSBC alone mutates authoritative memory and is therefore RTMA, not strict RTT.

Atomic Memory Rows supplies the complete-row/NULL authority substrate, while Predictive Memory Admission defines a separate future-risk decision problem [10, 11]. CSBC event construction does not validate either dependency. It can end in NULL under a fixed nonlearned rule, and this paper reports neither an oracle gap nor a learned admission policy.

Mechanism	Bidirectional/future access	Updated carrier	Why it is not CSBC by itself
BERT encoder	both sides of supplied input	representation/parameters during training	no causal-emission receipts or post-closure authoritative commit
Bidirectional RNN	forward/backward passes on supplied sequence	numerical hidden representation	no future-only exposure/authority contract
Fixed-interval smoothing	later observations in interval	latent-state estimate	offline estimator, not row eventization
BPTT	future losses in training graph	parameters/gradients	supervision clock, not deployment memory event
Transformer-XL	past segment recurrence	cached numerical context	causal recurrence, not completed-segment row authority
Compressive Transformer	compressed past activations	numerical memory	no typed trusted row or exact NULL
Offline consolidation / CLS	later/offline replay or integration	biological/model memory systems	conceptual analogy; no CSBC system receipt
TTT / Tent	test inputs/objective	numerical policy/model state	parameter adaptation, not authoritative memory
Strict RTT	prior feedback in unresolved problem	policy state Φ	CSBC lacks the required policy-update path
RTMA	causal memory mutation and later read	authoritative memory M	CSBC is one constrained RTMA protocol

Table 5: Category boundaries. The table is classificatory and does not assert priority, superiority, or empirical equivalence.

13.4 Narrow contribution

The paper's novelty claim is deliberately conditional and local: it assembles completed-view timing, five clocks, post-boundary exposure, prefix-twin noninterference, teacher amputation, bounded cross-boundary carry, stable ownership/idempotence, event-recursive authority, asynchronous pinning, and full resource obligations into one falsifiable CSBC contract. Novelty and related-work interpretations remain subject to independent scholarly evaluation.

14 Staged Falsification and Theorem-to-Receipt Obligations

No protocol below has been executed. Passing one stage permits only the next prespecified test; it does not validate a semantic interface or downstream system.

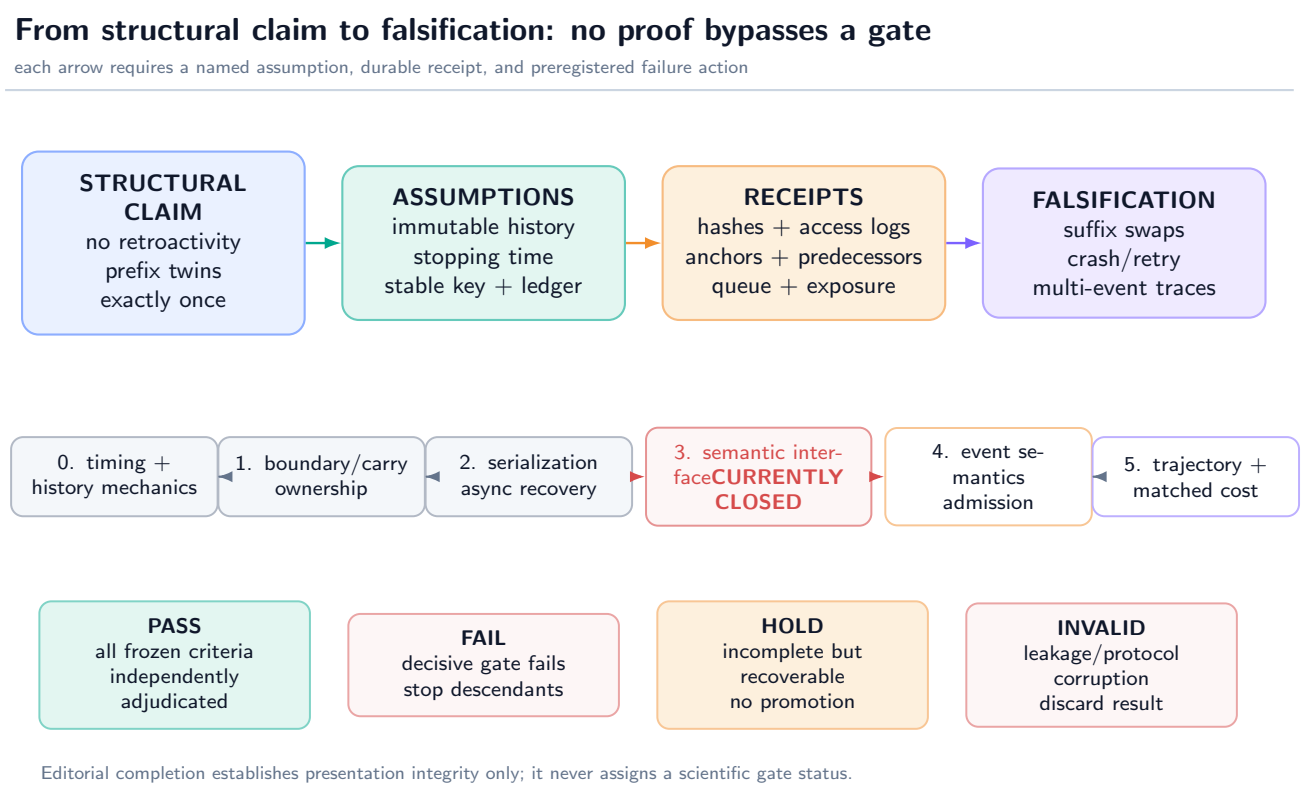

Figure 10: **Claim–assumption–receipt–falsification map and stop ladder (PLANNED TEST).** Structural proofs require premise receipts. Semantic qualification, predictive admission, long-run utility, and matched system cost are later independent gates. The current evidence chain remains stopped at the inherited semantic interface.

14.1 Structural timing gates

Protocol 14.1 (Prefix twins and suffix swaps)**.** Create paired executions with byte-identical prefix, prior state, build artifacts, boundary state, and random coins; hide two different suffixes until after the tested point. **Pass:** every pre-suffix boundary, view availability bit, event output, key, candidate, and exposure decision is byte-identical. **Fail/stop:** any difference before causal observation invalidates the timing implementation and every descendant result.

Protocol 14.2 (Historical-hash immutability)**.** Hash logits, coins, tokens, read versions, emission receipts, displayed-output history, and audit roots through each boundary. Inject eventization, validation, commit, retry, crash, and recovery. **Pass:** every pre-boundary hash remains identical. **Fail/stop:** one historical mutation voids the no-retroactive claim.

Protocol 14.3 (Boundary, overlap, carry, and ownership)**.** Generate fixed/adaptive cores at minimum/maximum lengths; events entirely inside a core, across one/two/many boundaries, at overlap edges, unresolved to the carry cap, and with adversarial tenant changes. **Pass:** causal stopping, finite views, bounded carry, one stable anchor/owner/key, typed overflow, and zero cross-tenant bytes. **Fail/stop:** duplicate/missing owner, suffix-dependent closure, silent truncation, unbounded carry, or tenant crossing.

14.2 Authority and service gates

Protocol 14.4 (Multi-event serialization and idempotence)**.** Create segments with $K_n = 0, 1, K_{\max}$, repeated targets, disjoint targets with and without shared reads, accepted events, NULL, stale workers, reordered completion, crash points, and repeated retries. **Pass:** lexicographic predecessors, at most one row per event, segment support $\leq K_n$, identical retry terminal, and old/new recovery. **Fail/stop:** worker-order semantics, duplicate terminal, partial row, lost prefix, or undeclared commutation.

Protocol 14.5 (Pinned asynchronous exposure)**.** Exercise stall and prior-state policies, timeouts, backlog, cache/index lag, and speculative decoding. **Pass:** every token binds one validated version; no successor is read before exposure; discarded speculation is never externally sealed; queue and retry costs reconcile. **Fail/stop:** mixed version, backdated receipt, partial candidate read, or silent drop.

Protocol 14.6 (Teacher amputation)**.** Freeze parameters and manifests before final access. Deny teacher namespaces at runtime; trace filesystem, database, cache, network, and feature calls; run label permutation and matched suffix swaps. **Pass:** no transitive teacher/future dependency and negative controls remain in the preregistered null range. **Fail/stop:** invalidate the model and all downstream semantic/utility claims.

14.3 Semantic and system gates

Protocol 14.7 (Semantic interface qualification)**.** Only after structural gates, test the same frozen checkpoint on canonical reconstruction, neural readout, fresh composition, query accuracy, calibration, stale suppression, unrelated-row preservation, routing consistency, corruption rejection, and all required seeds. **Pass:** every preregistered interface gate passes. **Fail/stop:** no CSBC semantic or memory-use claim. The frozen record currently fails before this stage.

Protocol 14.8 (Event semantics and predictive admission)**.** On fresh episode groups, separately test event precision/recall, anchor/ownership stability, target-conditioned row validity, and factual/source policy. Only after a qualified interface may the oracle-gap and future-blind admission ladder run. **Fail/stop:** keep fixed NULL or nonlearned behavior; no learned overwrite claim.

Protocol 14.9 (Long trajectories and recovery)**.** Run repeated segments through capacity, version, retention, deletion, rollback, archive/index, queue, and failure boundaries. Report stale leakage, contamination, missing/duplicate events, severe security outcomes, repair duration, and every carrier byte. **Fail/stop:** any severe invariant violation or unbounded hidden carrier blocks deployment claims.

Protocol 14.10 (Matched system cost and utility)**.** Compare causal-context, recurrent/compressive, causal-only segment encoder, CSBC with fixed rules, and any later qualified admission system under equal model, context, resident bytes, archive access, event horizon, attempt budget, hardware, and evaluator. Report the full vector in Eq. (27). **Pass:** a preregistered Pareto/materiality rule passes every seed and subgroup without a severe failure. **Fail/stop:** report the tradeoff; no quality–cost crossover or superiority claim.

14.4 Qualification statuses

Each stage ends as PASS under every frozen criterion, FAIL under a decisive criterion, HOLD for incomplete recoverable evidence, or INVALID for corrupted identification. Editorial completion cannot mark a scientific gate passed.

Result	Minimum premise receipt	Failure consequence
No retroactive effect	full history-carrier inventory, immutable hashes, exposure/read logs	theorem not implemented
Prefix-twin noninterference	stopping-time dataflow, matched coins/state, suffix deny log	causal boundary invalid
Teacher amputation	split closure, freeze receipt, transitive dependency/access graph	deployment model leaked
Finite views/work	boundary caps, carry bytes, archive/index/retry inventories	boundedness claim prohibited
Exactly once	stable anchor/key, owner proof, linearizable retained ledger, liveness	duplicate/missing claim unavailable
Event locality	whole-process authoritative carrier inventory and row diffs	hidden mutation invalidates support bound
Serializable history	logical sequencer, predecessor/read sets, stale rejection, global checks	worker schedule may change semantics
Conditional commutation	both-order replay and invariant candidate/decision/read sets	order is a treatment
Async consistency	one-root pin, cache/index binding, exposure and speculative receipts	token history ambiguous
Queue stability	arrival/service envelopes, worker availability, retry cap	no backlog/wait bound
Protected-field isolation	credential topology, negative authorization tests, trusted assembly audit	security proposition unusable
Semantic/utility/cost	independent later empirical protocols	no system claim from structural proof

Table 6: Theorem-to-implementation summary. Appendix B gives the complete obligation matrix.

15 Limitations, Explicit Nonclaims, and Conclusion

15.1 Limitations

1. The manuscript is theory and protocol only. No eventizer, encoder, queue, row assembler, storage engine, or model was implemented or evaluated.
2. A bounded completed view can omit context needed for a correct event. Larger overlap/carry changes cost and still cannot represent every unbounded dependency.
3. Adaptive boundaries are safe only when they are stopping times over observed data. Real detectors and distributed clocks can violate that premise.
4. Exactly-once is scoped to one event schema, episode/retry horizon, stable key, and retained linearizable ledger. It is not eternal global deduplication.

5. Mechanical exactly-once publication is compatible with a semantically wrong, malicious, obsolete, or unauthorized proposal that a defective trusted system accepts.
6. Atomicity is per event. A segment can partially succeed, and a multi-row invariant requires an additional transaction protocol.
7. The serializability and commutation results depend on complete read sets and global constraints. In practice, capacity, indexes, caches, policies, or allocators can be hidden shared state.
8. Queue stability uses deterministic envelopes. Heavy-tailed service, failures, shared accelerators, and correlated bursts require a stronger model.
9. The resource bounds exclude any undeclared full-history archive scan, unbounded ANN graph, unbounded retry, or hidden provenance carrier. Those omissions invalidate the theorem rather than becoming constants.
10. Teacher amputation depends on a complete dependency graph and source-family split. Runtime field names alone cannot prove the absence of transitive leakage.
11. Trusted assembly is a large trust boundary. Correct cryptography or atomic storage does not establish policy correctness, factual truth, privacy, or freedom from side channels.
12. A later correction cannot erase harm caused by an already emitted statement. Historical integrity and semantic remediation are different objectives.

15.2 Explicit nonclaims

This paper does not claim that:

- the frozen same-checkpoint semantic interface is qualified;
- CSBC extracts correct events or corrections in practice;
- any authoritative row can be usefully read by the same checkpoint;
- an expected-risk oracle gap or predictive overwrite exists;
- a learned admission, routing, eventization, or value policy exists;
- post-closure bidirectionality changes an earlier logit, token, receipt, or displayed history;
- CSBC alone is strict RTT or online parameter learning;
- fixed or adaptive segmentation is optimal;
- exactly-once mechanics imply complete recall, truth, safety, or deletion;
- asynchronous execution is linearizable without the stated sequencer and storage premises;
- bounded resident memory implies bounded archive, index, audit, training, or total-system storage;
- any latency, energy, hardware, quality, or cost crossover has been measured; or
- completing the formal family changes any empirical gate or workflow state.

15.3 Conclusion

The essential separation is temporal. Generation through b_n is causal and immutable. A completed view exists only after b_n. Retrospective encoding may then use both directions inside that bounded observed view. Ordered events can produce complete authoritative successors, but each successor has a later exposure point $a_{n,j} > b_n$. The past remains a receipt, not an editable buffer.

That separation becomes auditable only after the rest of the contract is made explicit: five clocks; stopping-time boundaries; overlap and bounded carry; stable anchors, ownership, and idempotence; event-recursive hard commit or exact NULL; a logical sequencer independent of worker finish order; pinned asynchronous reads; teacher amputation; complete authority isolation; and a resource ledger that includes the work outside the encoder.

Under those premises, the structural results are useful and narrow. They rule out retroactive effects, suffix-dependent pre-observation behavior, duplicate terminal outcomes, hidden segment-wide atomicity, and free asynchronous service. They do not show that the resulting memory is correct or helpful. The current empirical chain remains stopped at the inherited semantic-interface gate. CSBC is therefore a falsifiable causal and systems contract for reasoning-time memory adaptation, not an empirical memory or reasoning result.

Author Contributions

Junyi Zou and Avrova Donz contributed equally to conceptualization, formal analysis, methodology, visualization design, and manuscript preparation. No corresponding-author designation is made.

AI-Assisted Tooling Disclosure

AI-assisted programming and writing tools supported drafting, mathematical consistency checks, vector-figure generation, citation verification, build diagnostics, and visual quality assurance under human direction. The named authors retain responsibility for the manuscript. AI output and editorial completion are not scientific evidence.

References

[1] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988. Association for Computational Linguistics, 2019.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186. Association for Computational Linguistics, 2019.

[3] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.

[4] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[5] Herbert E. Rauch, F. Tung, and Charlotte T. Striebel. Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8):1445–1450, 1965.

[6] Mike Schuster and Kuldip K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997.

[7] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522, 2025.

[8] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021.

[9] Paul J. Werbos. Backpropagation through time: What it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.

[10] Junyi Zou and Avrova Donz. Atomic memory rows: A bounded, verifiable substrate for editable reasoning. MMLA Focus Paper; cited for the formal row contract, 2026.

[11] Junyi Zou and Avrova Donz. Learning what to remember: Predictive admission for bounded reasoning-time memory. MMLA Focus Paper; cited for the formal admission problem, 2026.

[12] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future. *arXiv preprint arXiv:2606.28876v3*, 2026.

[13] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future, complete technical report. Technical report, MMLA-org, 2026. Complete technical-report evidence record.

[14] Junyi Zou and Avrova Donz. MMLA-RTT: Dual-state learning at reasoning time. MMLA Focus Paper; cited for typed dual-state semantics, 2026.

[15] Junyi Zou and Avrova Donz. Reasoning-time training: Learning before a single problem ends. MMLA Focus Paper; cited for formal terminology, 2026.

A Supplementary Proofs and Formal Boundaries

All results in this appendix are deductions from displayed premises. They do not establish implementation conformance or empirical usefulness.

A.1 Stopping-time prefix closure

PROVED UNDER STATED ASSUMPTIONS

Lemma A.1 (Boundary decisions are prefix functions)**.** *Under Assumption 6.1, whether adaptive boundary b_n has occurred by token u is determined by the observed prefix through u. Two u-prefix twins therefore agree on all boundary decisions through u.*

Proof. The stopping-time property is precisely $\{b_n \le u\} \in \mathcal{F}_u^X$. Prefix twins agree on the realization of every generator of $\mathcal{F}_u^X$ named by the boundary rule, including detector state and revealed coins. Hence the indicator of $\{b_n \le u\}$ agrees. Apply this to each boundary whose search interval begins by u. □

The lemma fails if the implementation uses a future token, future-derived feature, retrospective encoder output produced from a larger view, or wall-clock worker completion correlated with hidden suffix processing.

A.2 Exposure monotonicity

PROVED UNDER STATED ASSUMPTIONS

Proposition A.2 (History-prefix preservation under composition)**.** *Let $T_1, \dots, T_m$ be any finite sequence of conforming post-closure service, candidate, validation, commit, retry, recovery, and exposure transitions. Under Assumption 2.1, their composition restricts to the identity on every sealed history object that predates the first transition.*

Proof. Each transition has the identity map on the sealed history coordinates by Assumption 2.1. The composition of identity restrictions is identity. The transitions may append new receipts, but append does not change the earlier prefix. □

PROVED UNDER STATED ASSUMPTIONS

Corollary A.3 (A later correction is a new causal fact)**.** *If a CSBC event represents a correction to an earlier emitted assertion, then the correction's authoritative effect, if any, is a new later state transition. It is not a modification of the earlier assertion or its emission receipt.*

Proof. The earlier assertion and receipt belong to the sealed history prefix and are fixed by Proposition A.2. Any accepted correction appears only in the new authoritative successor and later reads. □

A.3 Ownership and terminal-state uniqueness

PROVED UNDER STATED ASSUMPTIONS

Lemma A.4 (Core partition yields unique owner)**.** *If $0 = b_0 < b_1 < \cdots < b_B = N$ and $I_n = (b_{n-1}, b_n]$, then every token anchor $a \in \{1, \dots, N\}$ belongs to exactly one core.*

Proof. The increasing boundary sequence partitions $\{1, \dots, N\}$ into disjoint half-open integer intervals. Existence follows from $b_B = N$; uniqueness follows because two distinct intervals share no integer. □

Overlap does not alter the lemma because O_n is not part of the ownership partition.

PROVED UNDER STATED ASSUMPTIONS

Proposition A.5 (Terminal idempotence)**.** *Under Assumption 7.3, after key ι has a terminal record, any finite number of retries with that key returns the same terminal authoritative outcome and makes no additional authoritative mutation.*

Proof. The linearizable ledger transition graph has no outgoing mutation edge from a terminal state except a read of that state. A retry cannot claim the key as absent, so it cannot invoke a second authoritative publication. Induction over retries gives the result. □

If the ledger is garbage-collected before the maximum retry horizon, the premise no longer holds. A digest in an unauthenticated log is not a linearizable idempotence ledger.

A.4 Event order and commutation

PROVED UNDER STATED ASSUMPTIONS

Proposition A.6 (Repeated-target recursion)**.** *If two ordered events target the same row and both commit, the second candidate must validate against the first successor. Reversing the events generally changes the final row even when both candidates are individually valid against the initial state.*

Proof. Equation (21) sets the second predecessor to $M_{n,1}$. Monotone version, predecessor receipt, rollback lineage, and possibly semantic content therefore depend on the first transition. In the reversed order, those fields and the final payload can differ. No commutation assumption applies because targets and read/write sets overlap. □

PROVED UNDER STATED ASSUMPTIONS

Proposition A.7 (NULL is not an event omission)**.** *A terminal NULL decision is distinguishable from an event that was never emitted, a worker that remains in flight, and an event dropped by carry overflow.*

Proof. Terminal NULL has a claimed stable key, logical event index, predecessor, validation/decision receipt, and identity authoritative transition. A missing event has no such event key; an in-flight event has no terminal state; an overflow has a detector failure receipt before commit. The typed ledgers therefore distinguish all four cases. □

A.5 Queue and exposure consequences

PROVED UNDER STATED ASSUMPTIONS

Corollary A.8 (Finite additional exposure lag under stable service)**.** *Under Theorem 10.3, finite per-job encode/event/candidate/validation/publication bounds, and a work-conserving sequencer, the wall-clock lag from segment closure to terminal publication is finite for every admitted job.*

Proof. The queue theorem gives a finite wait bound. The remaining service stages are a finite sum of finite bounds. Sequencer waiting is included in the declared service envelope. Their sum is finite. □

The corollary does not bound token-index exposure if the deployment waits for another semantic pin point, nor does it hold for jobs intentionally dropped by admission control.

A.6 Proof-dependency graph

The logical dependencies are:

$$
\begin{aligned}
&\text{immutable history + post-closure exposure} \Longrightarrow \text{no retroactive effect},\\
&\text{stopping-time boundary + deterministic prefix computation} \Longrightarrow \text{prefix-twin noninterference},\\
&\text{bounded cores/overlap/carry/events/targets} \Longrightarrow \text{finite work and memory},\\
&\text{stable anchor + unique core owner + linearizable key ledger} \Longrightarrow \text{at-most-once terminal},\\
&\text{at-most-once + recoverable live owning job} \Longrightarrow \text{exactly-once terminal},\\
&\text{complete-row/NULL event + recursion} \Longrightarrow \text{event and segment support},\\
&\text{predecessor/read-set validation + logical sequencer} \Longrightarrow \text{serializable authority},\\
&\text{serial authority + exposure selector + one-root pin} \Longrightarrow \text{asynchronous causal consistency},\\
&\text{split independence + complete amputated graph} \Longrightarrow \text{future-teacher nonaccess}.
\end{aligned}
$$

No arrow reverses. Observed exactly-once behavior cannot prove stable anchors; a clean prefix-twin test cannot prove semantic correctness; a serializable commit cannot prove no future leakage.

B Complete Theorem-to-Implementation Obligation Matrix

Table 7: A conditional result becomes an implementation statement only after every premise in its row has a durable receipt.

Object/result	Formal premise	Minimum durable evidence	Failure consequence
Causal token	$\mathcal{D}_t$-measurable logits/sample	input/version/coin/logit/token receipt; future-access guard	generation causality unsupported

Continued on next page

Object/result	Formal premise	Minimum durable evidence	Failure consequence
Immutable history	append-only complete carrier inventory	before/after hashes for logs, UI, receipts, caches, replicas	no-retroactive theorem inapplicable
Completed view	constructed only after sealed b_n	boundary token/receipt, view start time, exact source-token digest	"completed" status ambiguous
Fixed boundary	declared C, O and episode end	boundary manifest and rerun equality	coverage/overlap counts unavailable
Adaptive boundary	stopping time; $C_{\min}, C_{\max}$	static dataflow, runtime prefix access, min/max exhaustion tests	prefix noninterference invalid
Bounded carry	B_Q, H_Q and typed overflow	serialized byte/age counters, adversarial open-event tests	memory/work theorem invalid
Retrospective encoder	only completed view/carry; frozen build	input manifest, bidirectional graph, build hash, start-after-close receipt	temporal scope unverified
Prefix twins	identical prefix/state/build/coins	paired suffix-swap manifests and byte-level output comparison	future leakage not excluded
Teacher amputation	split independence; complete dependency closure	source-family split, freeze receipt, transitive provenance and deny/access logs	model invalid for deployment claim
Stable event	deterministic normalization/detector	repeated boundary/retry/resegmentation equality	event identity unstable
Unique anchor/owner	anchor in one core; overlap non-owning	anchor and core partition proof per event	duplicate or missing event possible
Idempotence	linearizable retained key ledger	claim/terminal history, split-brain tests, retention horizon	at-most-once unavailable
Exactly-once liveness	owning job eventually terminal/recoverable	queue lease, retry/recovery terminal and timeout policy	only at-most-once may be claimed
Complete row/NULL	trusted assembly; exact identity reject	full row validation, authoritative before/after diff, NULL equality	event-locality claim invalid
Event recursion	actual predecessor per event	ordered predecessor/candidate/successor digest chain	later event uses stale branch
Segment support	all authority inventoried in M	process-wide writes plus per-row hashes	hidden carrier defeats bound
Serializability	logical total order; complete read/global checks	linearization and stale-reject receipts; write-skew tests	worker schedule may alter result
Conditional commutation	disjoint writes and invariant complete read sets	both-order replay, same candidates/decisions/versions	order must remain semantic treatment
One-root pin	immutable validated root per token	read-service/cache/index version receipt	mixed-state generation possible
Exposure	$a_{n,j} > b_n$ and published/validated selector	token-index and wall-time exposure manifest	future-only influence unsupported
Speculation	commit-old, discard/regenerate, or stall	draft/seal/discard/cache receipts and hashes	historical version ambiguous
Queue stability	arrival/service envelopes and worker guarantee	timestamp trace, max-job bound, retry/burst accounting	backlog/latency bound unavailable
Finite work/memory	all caps and carriers complete	allocator, archive/index, scratch, retry, traffic inventories	asymptotic claim incomplete
Trusted fields	least privilege and system derivation	credentials, code/dataflow audit, adversarial field injection	authority/security result unavailable
Crash recovery	old/new atomic root and audit binding	crash injection at every durable boundary	partial/mixed state possible
Deletion/rollback	current authorization and every carrier covered	stale replay, cache/index/archive/replica deletion tests	resurrection risk unresolved
Semantic interface	independent same-checkpoint gates	reconstruction, query, composition, preservation, integrity receipts	no CSBC semantic-use claim
Event correctness	fixed schema and source/truth evaluation	fresh precision/recall, calibration, adversarial source tests	mechanics only
Predictive admission	qualified interface and predictive-admission ladder	oracle-gap, future-blind value, uncertainty/NULL receipts	no learned overwrite/policy claim
Strict RTT	feedback-driven Φ update and later same-problem read	RTT and dual-state timing/intervention receipts	CSBC remains RTMA only
System advantage	matched complete quality/cost study	all vector costs, baselines, seeds, subgroups, severe events	no superiority or crossover claim

B.1 Minimal event receipt schema

For auditability, every event record contains at least

$$\begin{aligned} R^{\text{event}}_{n,j} = (&\mathsf{episode}, n, j, b_n, \alpha, \iota, \mathsf{owner}, \mathrm{Hash}(V_n), \mathrm{Hash}(q_{n-1}), \mathrm{Hash}(e_{n,j}), \\ &\mathsf{builds}, \mathsf{coins}, \nu_{\text{pred}}, \mathsf{readset}, \mathsf{candidate\text{-}digest}, \mathsf{validation}, \mathsf{action}, \nu_{\text{succ}}, a_{n,j}, \\ &\kappa^{\text{enq}}, \kappa^{\text{start}}, \kappa^{\text{done}}, \kappa^{\text{pub}}, \kappa^{\text{exp}}, \mathbf{C}_{n,j}). \end{aligned} \tag{35}$$

A field can be a bounded authenticated digest or typed absence marker. Missing is never silently converted to zero, NULL, no event, or no cost.

B.2 State-machine conformance sketch

The permitted key states are

$$\mathsf{ABSENT} \to \mathsf{INFLIGHT} \to \{\mathsf{COMMITTED}(\nu), \mathsf{NULL}, \mathsf{FAILED\_TERMINAL}\}. \tag{36}$$

Transient worker failures leave the key INFLIGHT with a renewable bounded lease and appended attempt receipt. A protocol may choose FAILED_TERMINAL for unrecoverable detector/authorization/system errors; that state is not authoritative NULL and cannot be reported as an admission decision.

C Counterexample, Failure, and Audit Catalogue

C.1 Boundary and leakage catalogue

Witness	Violated premise	Observable audit	Required status
next-token peek	stopping-time boundary	prefix-twin boundary mismatch	INVALID timing result
teacher-derived feature cache	graph closure/split independence	provenance ancestor or suffix-swap change	INVALID model
late UI rewrite	complete immutable-history inventory	displayed-history hash changes	FAIL no-retro implementation
future-dependent timeout	causal exposure policy	matched queue state, differing suffix changes fallback	INVALID causal decision
cross-episode ID collision	episode-bound stable key	same digest under distinct episode authority	FAIL idempotence scope

Table 8: Temporal leakage witnesses and their direct audits.

C.2 Event and authority catalogue

Witness	Violated premise	Observable audit	Required status
overlap emits twice	unique owner	same normalized event from two core owners	FAIL exactly-once
wall-time key	stable idempotence identity	retry creates second terminal	FAIL at-most-once
unbounded open quotation	carry cap	serialized carry grows with boundary count	FAIL boundedness
partial row publish	atomic complete-row authority	mixed field/version receipt	severe FAIL
proposal self-assigns ACL	least privilege	adversarial field injection succeeds	severe FAIL
stale worker commit	sequencer/predecessor check	accepted candidate names obsolete root	FAIL serializability
shared archive capacity	commutation independence	both-order decisions differ	order is required treatment
semantic poisoning	truth outside mechanics	mechanically valid but false row	semantic/safety FAIL, mechanics separately reported

Table 9: Event, transaction, and authority witnesses.

C.3 Hidden-state and hidden-cost audit

DESIGN PRESCRIPTION A future conformance review inventories all of the following, including zero/absent declarations:

1. generator parameters, adapters, optimizer/TTT state, sampler seeds, KV caches, speculative buffers, and visible context;
2. boundary detector state, overlap buffers, carry, retrospective activations, event queues, candidate buffers, validation caches, and idempotence keys;
3. authoritative rows, versions, ACL/policy state, tombstones, rollback snapshots, commit roots, and derived indexes;
4. archives, retrieval indexes, source documents, provenance graphs, audit ledgers, replicas, backups, filesystem/journal overhead, and deletion carriers;
5. training futures, labels, feature stores, preprocessing caches, split metadata, calibration thresholds, checkpoints, and evaluation state;
6. CPU/accelerator work, memory/network/storage traffic, barriers, failed/retried work, queue wait, wall time, and energy measurement/proxy.

For each carrier the inventory gives capacity, authority, writer, reader, lifetime, reset, deletion, version, cost unit, and receipt. Calling a carrier "cache," "metadata," or "temporary" does not remove its causal or resource role.

C.4 Worked event trace

Consider a completed core containing: "The package is in Rome. Correction: it is in Milan." The example is illustrative only.

1. Token receipts seal both sentences under prior memory version ν_4.
2. Boundary b_n closes after the period. The view contains the bounded core and declared overlap.
3. The eventizer proposes one correction event anchored at the observed token "Milan" and key ι.
4. Trusted assembly checks tenant, object identity, source receipt, ACL, current version, protection, and canonical/neural profile. It may reject with NULL.
5. If accepted in logical event order, the row publishes as ν_5 and receives exposure $a_{n,1} > b_n$.
6. A later question that pins ν_5 may read Milan. The two earlier sentences and both emission receipts remain unchanged.

The trace demonstrates the contract's chronology, not that an eventizer will recognize the correction or that Milan is true.

D Symbol, Status, and Evidence Ledger

D.1 Core symbols

Table 10: Primary notation.

Symbol	Type	Meaning
$t, n, (n, j), s, \kappa$	five clocks	token, boundary, logical event, training-supervision, wall service time
X_t, L_t, ξ_t	token/logits/coin	causal sample at token time t
ρ_t^X	immutable receipt	binds prefix, logit, coin, token, read version, workspace
b_n, I_n, C_n	boundary/core	terminal core index, half-open core, core length
O_n	bounded token set	read-only overlap preceding core I_n
$q_n \in \mathcal{Q}$	bounded bytes	unfinished cross-boundary detector carry
$\beta_n \in \mathcal{B}$	rule artifact	fixed or adaptive causal boundary rule
$V_n \in \mathcal{V}$	completed view	overlap, core, and prior carry after b_n
B_ψ	frozen map	post-closure bidirectional/retrospective encoder
$E_n, e_{n,j}, K_n$	event list/event/count	deterministic ordered events for segment n
$\alpha(e)$	token index	stable event anchor
$\mathrm{Owner}(e)$	boundary index	unique core containing the anchor
$\iota(e)$	authenticated digest	retry/idempotence identity
$\widehat{r}_{n,j}$	complete row or $\perp$	untrusted target-conditioned candidate
$M_{n,j}, \nu$	bounded state/version	authoritative memory after event j
NULL	terminal action	bit-identical authoritative identity transition
$a_{n,j}$	token index	earliest declared exposure of successor event state
Q^{svc}	bounded queue	asynchronous consolidation jobs
F, Y^{T}	training-only	future branches and teacher labels
θ, ψ	frozen parameters	generator and retrospective encoder artifacts
$\mathbf{C}$	nonnegative vector	complete resource ledger

D.2 Status ledger

Object	Status in this paper	Boundary
Typed CSBC timing/system contract	DEFINITION	new editorial mathematical construction
No-retro, prefix-twin, amputation, ownership, recursion, async, and resource results	PROVED UNDER STATED ASSUMPTIONS	conditional on displayed premises
Fixed/adaptive protocols and receipts	DESIGN PRESCRIPTION	no implementation exists
All falsification ladders	PLANNED TEST	no stage executed
Same-checkpoint semantic interface	UNVALIDATED	inherited failed/unqualified gate
Event extraction correctness	UNVALIDATED	no data or eventizer test
Predictive overwrite/admission	UNVALIDATED	no oracle gap or learned policy
Strict RTT	UNVALIDATED	no feedback-driven policy update path
CSBC utility, safety, and efficiency	UNVALIDATED	no empirical or systems result

Table 11: Manuscript completion does not promote any scientific status.

D.3 Frozen numerical boundary

Only inherited negative/diagnostic counts are admitted:

- PM-I2: 0/9 registered interfaces qualified.
- PM-I3 direct candidate: 40/73,728 exact; direct present: 0.
- Learned route/learned content: 0/73,728.
- Oracle route/learned content: 0/73,728.
- Learned route/oracle content: 18,544/73,728, exactly the routing count.
- Oracle route/oracle content: 73,728/73,728, a mechanical ceiling only.
- Preservation: 0; missing cases: 43,008.

These values delimit what is not qualified. They estimate no CSBC boundary, event, queue, memory transition, semantic accuracy, latency, energy, or cost.